A Handbook

Automated Student Grading System
(FYP Report)

Supervisor: Muhammad Ans Khalid



Submitted by:

Usama Shoaib (BSCS-023A21-15)

Department of Computer Science & IT
University of Southern Punjab, Multan 2025

AUTHOR'S DECLARATION

I hereby declare that I am the sole author of this FYP Project (Thesis). This is a true copy of the thesis, including any required final revisions, as accepted by my examiners. It is further declared that this thesis or any part thereof has not been previously submitted for any degree or diploma in any university or other educational institution.

I understand that my thesis may be made electronically available to the public.

Student Name	Usama Shoaib
Student Signature:	Usama Shoaib
Registration No.	BSCS-023A21-15
Email Address	Usamashoaibkhan22@gmail.com
Contact No.	0330-6787675
Year of Study	(2021-2025)
Area of Study	BSCS
Proposed Project Title	Automated Student Grading System
Discipline	Computer Science

FYP Head Signature/Stamp:

Name:Dr. Muhammad Azam
Contact Number: +92 333 8556894
Email: muhammadazam@usp.edu.pk

UNIVERSITY OF SOUTHERN PUNJAB

Supervisor Signature:

Name: Muhammad Ans Khalid
Contact Number: +92 304 7117145
Email: anskhalid@usp.edu.pk

University of Southern punjab Multan, Pakistan
Department of Computer Science

Acknowledgement

I would like to express my sincere gratitude to **Dr. Mubashir Malik**, **Head of the Computer Science Department** at the University of Southern Punjab, Multan, for his invaluable guidance, encouragement, and continuous support throughout the project. His leadership created a motivating academic environment that inspired me to strive for excellence.

I am also deeply thankful to **Dr. Muhammad Azam**, **FYDP Project Head** at the University of Southern Punjab, Multan, for his constant support, constructive feedback, and encouragement throughout the project. His guidance was instrumental in refining the design and implementation, and in ensuring the overall success of the work.

I would like to express my sincere gratitude to my project supervisor, **Sir Muhammad Ans Khalid** from the Computer Science Department, University of Southern Punjab, Multan, for his continuous guidance and dedicated monitoring of the project. His valuable insights, particularly in the areas of Natural Language Processing (NLP) and data extraction, greatly assisted me in understanding the technical aspects and in developing the core functionalities of the Automated Student Grading System. His mentorship ensured that the project progressed in the right direction and met its intended objectives.

I put in significant effort, dedication, and problem-solving to ensure that the project was meaningful, impactful, and successfully achieved its objectives. Lastly, I would like to thank my faculty for their encouragement, moral support, and patience, which motivated me to stay focused and determined throughout the entire project journey.

Project done by:

Usama Shoaib.

Abstract

This Automated Student Grading System is developed using ASP.NET Core (MVC architecture) in C# with Visual Studio 2022, integrating a Microsoft SQL Server 2008 backend and a web-based frontend built with HTML, CSS, and JavaScript. The system is designed to streamline the process of evaluating student assignments through a secure, role-based online platform. It reduces manual grading efforts, ensures consistency in evaluation, and provides timely, data-driven feedback to students.

The platform supports multiple user roles including Admin, Instructor, and Student, each with secure login and personalized dashboards. Instructors can manage courses, create assignments, grade submissions, and provide feedback, while students can submit work digitally, track progress, and review performance history.

At its core, the system integrates an NLP-based grading module powered by DeepInfra's large language models (e.g., Meta-LLaMA 3 8B Instruct). Assignments are processed through API requests where the model evaluates content relevance, grammar, and structure to predict grades, identify at-risk students, and generate performance summaries. These automated evaluations complement manual grading, offering both accuracy and efficiency.

Additionally, instructors and administrators can generate detailed reports, analyze predicted vs. actual grades, and monitor academic trends. Students benefit from transparent performance tracking, notifications, and personalized feedback, promoting continuous learning.

By combining secure access control, structured course management, and advanced NLP-powered automation, the system demonstrates significant potential to reduce grading time, enhance fairness, and improve overall academic quality assurance.

Table of Contents

1. Introduction **1**
1.1 Background 1
1.1.1 Context and Motivation 1
1.1.2 System Overview and Technical Significance 1
1.1.3 Relevance and Impact in the Educational Field 2
1.2 Problem Statement 3
1.3 Objectives 3
1.4 Scope 4
1.5 Significance 4
2. Literature Review **6**
2.1 Review of Related Work 6
2.1.1 Previous System 1: Student Grade Prediction System 6
2.1.2 Previous System 2: Student Grading System (SGS) 9
2.1.3 Summary of Limitations in Existing Systems 12
2.2 Gap Analysis 13
2.2.1 Identified Gaps in Previous Systems 13
2.2.2 Proposed Solution: Automated Student Grading System 14
2.2.3 Comparison Table 15
3. Methodology **17**
3.1 Project Design 17
3.1.1 System Architecture 17
3.1.2 Workflow Diagram 17
3.1.4 Data Flow 19
3.2 Technology Stack 20
3.2.1 Programming Languages 20
3.2.2 Frameworks & Libraries 20
3.2.3 Database 20
3.2.4 Tools & Platforms 20
3.2.5 Hardware/Cloud Requirements 20
3.3 Implementation 20
3.3.1 Data Collection & Preprocessing 20
3.3.2 Model Development 21

3.3.3 System Integration 21
3.3.4 User Interface Development 22
3.4 Development Model (Spiral Model) 22
3.5 Testing 23
3.5.1 Testing Strategy 23
3.5.2 Test Cases 23
3.5.3 Performance & Security Testing 24
3.5.4 User Acceptance Testing (UAT) 24
3.5.5 Results & Analysis 24
4. System Requirements 25
4.1 Functional Requirements 25
4.2 Non-Functional Requirements 26
5. System Design 28
5.1 Design Overview 28
5.1.1 Use Case Diagrams 29
5.2 Architecture 33
5.2.1 System Architecture Diagram 34
5.2.2 Data Flow Diagrams (DFDs) 36
5.3 Data Design 38
5.3.1 Entity Relationship Diagram (ERD) 38
5.3.2 Database Schema 42
5.4 Interface Design 45
5.4.1 Admin Interfaces 45
5.4.2 Teacher Interfaces 54
5.4.3 Student Interfaces 66
6. Results and Discussion 71
6.1 Results 71
6.2 Discussion 76
7. Conclusion 78
8. References 79

List of Figure

Figure 1: DFD of previous system 1 8
Figure 2: GUI Architecture of previous system 1 8
Figure 3: DFD of previous system 2 11
Figure 4: Instructor view of previous system 2 11
Figure 5: Student view of previous system 2 12
Figure 6: Use Case Diagram for User Account Management System 30
Figure 7: Admin - Use Case Diagram 31
Figure 8: Teacher – Use Case Diagram 31
Figure 9: Student – Use Case Diagram 32
Figure 10: Automated Student Grading System – Use Case Diagram 33
Figure 11: System Architecture Diagram 35
Figure 12: (DFD) Level 0 of the Automated Student Grading System 36
Figure 13: (DFD) Level 1 of the Automated Student Grading System 37
Figure 14: ERD 42
Figure 15: Schema Diagram of Automated Student Grading System 45
Figure 16: User Authentication Screen 46
Figure 17: Admin Control Panel 47
Figure 18: Department Management Screen 48
Figure 19: Subject Management Screen 49
Figure 20: Semester-Wise Subject Screen 50
Figure 21: Teacher Management Screen 51
Figure 22: Student Management Screen 52
Figure 23: Result Management Screen 53
Figure 24: Detailed Result & View Screen 54
Figure 25: Teacher Authentication Screen 55
Figure 26: Teacher Dashboard Screen 56
Figure 27: Teacher Profile Update Screen 57
Figure 28: All Students Screen 57
Figure 29: Teacher Assignment Upload Screen 58
Figure 30: Student Submission Tracking Screen 59
Figure 31: All Students Screen 59
Figure 32: Assignment Auto-Grading Screen 60
Figure 33: Assignment Result & Feedback Screen 61
Figure 34: Result Entry & Grade Calculation Screen 62
Figure 35: Result Record & Search Screen 63
Figure 36: Detailed Student Subject Result Screen 64
Figure 37: At-Risk Students Monitoring Screen 65
Figure 38: Feedback Communication Screen 66
Figure 39: Student Authentication Screen 66
Figure 40: Profile Update Screen 67
Figure 41: Assignment Overview Screen 68
Figure 42: Assignment Upload Screen 68
Figure 43: Notification/Teacher's Chat Screen 69
Figure 44: Chat with Teacher Screen 69
Figure 45: All Results Screen 70

Figure 46: Result Details Screen..... 70
Figure 47: Login Screen of the System..... 71
Figure 48: Example of Invalid Login Attempt 72
Figure 49: Successful Dashboard Access for Different User Roles (Admin, Teacher, Student) . 72
Figure 50: Graphical View and GPA Distribution Graph 73
Figure 51: Student Assignment Submitted Before NLP Evaluation 74
Figure 52: Assignment Result Generated Through NLP Prediction and Evaluation 74
Figure 53: Complete Result View of Student..... 75
Figure 54: Teacher Feedback Sent to Student 76
Figure 55: Result of Teacher Feedback Viewed by Student..... 76

List of Tables

Table 1: Comparison of Previous Systems and Proposed System..... 15
Table 2: Strong and weak entities of ERD..... 42

1. Introduction

1.1 Background

This Automated Student Grading System is a web-based platform developed using ASP.NET MVC architecture to automate the process of evaluating student assignments. The system aims to reduce manual grading effort, enhance evaluation consistency, and provide timely, detailed feedback to students. It is designed for multiple user roles Admin, Instructor, and Student with each having specific features and permissions.

1.1.1 Context and Motivation

In the modern educational landscape, the volume of assignments and written assessments submitted by students has significantly increased due to the growing emphasis on project-based learning and continuous assessment methods. While these approaches enhance the quality of education, they also put considerable strain on instructors who must manually evaluate a large number of submissions. Manual grading is time-consuming, prone to human bias, and may lack consistency across different evaluators. In many institutions, delays in grading also result in delayed feedback to students, which can hinder their learning progress.

The Automated Student Grading System addresses this challenge by introducing a structured, technology-driven solution for assignment evaluation. By using ASP.NET MVC architecture, the system ensures a robust, maintainable, and scalable foundation for managing academic workflows. The project aims to automate grading without compromising accuracy, fairness, or the personal touch that educators bring. The motivation behind the system lies in bridging the gap between traditional manual assessment and the potential of Artificial Intelligence (AI) to improve educational outcomes.

1.1.2 System Overview and Technical Significance

The platform follows the Model-View-Controller (MVC) design pattern, which separates the application into three interconnected components:

- Models: These represent academic entities such as Student, Teacher, Department, Semester, Subject, and Predict Result, which form the backbone of the data layer.
- Views: The user interface components through which Admins, Instructors, and Students interact with the system.
- Controllers: Modules such as Student Controller, Teacher Controller, and Subject

Controller that handle incoming HTTP requests, process them using business logic, and return appropriate responses.

Secure login and authentication ensure that only authorized users access the platform. Instructors can manage courses, create and review assignments, while students can submit their work digitally. The core innovation lies in integrating DeepInfra's NLP API via DeepInfraService and NLP Service, enabling automated text analysis. This integration allows the system to process assignment submissions, assess their grammar, relevance, and structure, and predict grades. The predictions are stored in the database alongside instructor-provided grades, enabling comparison and further analysis. This combination of MVC architecture and NLP-driven grading brings technical significance to the project by merging web application best practices with AI-based evaluation. The result is a system that not only automates grading but also supports academic decision-making through accurate data insights.

1.1.3 Relevance and Impact in the Educational Field

The Automated Student Grading System holds substantial relevance for educational institutions, particularly universities like the Institute of Southern Punjab, Multan, where both instructors and students can benefit from faster, more consistent evaluation methods. For faculty, it significantly reduces the grading workload, enabling them to dedicate more time to personalized teaching and student mentorship. The consistency provided by the NLP model ensures that all submissions are evaluated against the same standards, reducing subjective discrepancies that often arise in manual grading.

For students, timely feedback is one of the most crucial elements in the learning process. With this system, feedback can be delivered almost instantly after submission, allowing students to identify and address their weaknesses before moving on to the next stage of their learning. Additionally, the system stores performance history and risk analysis, enabling long-term academic tracking and targeted interventions. In the broader educational field, this project aligns with global trends in EdTech (Educational Technology), where AI is increasingly being used to personalize learning, predict student performance, and optimize academic resources. The integration of a powerful NLP model for academic evaluation demonstrates how emerging technologies can be adapted to institutional needs while maintaining security, scalability, and user-friendliness.

Ultimately, the project was undertaken to solve a pressing problem manual grading inefficiency while simultaneously contributing to the advancement of AI-powered educational tools. It serves

as a practical example of how technology can be leveraged to enhance teaching effectiveness, improve student learning experiences, and modernize academic evaluation systems.

1.2 Problem Statement

In academic institutions, the grading of student assignments is a critical yet time-intensive process. Traditionally, instructors manually evaluate each submission, which can lead to delays in providing feedback, inconsistency in grading, and increased workload, especially when dealing with large classes. Human evaluation is also susceptible to subjective bias, which may affect the fairness and reliability of the results.

Although some automated grading tools exist, they are often limited in scope, costly to implement, or unable to integrate seamlessly with institutional academic management systems. Many of these solutions fail to provide detailed, personalized feedback or to analyze student performance trends over time.

The absence of a cost-effective, AI-powered grading platform that integrates secure authentication, course and assignment management, and advanced Natural Language Processing (NLP) capabilities represents a significant gap in current educational technology.

This project addresses that gap by developing a web-based Automated Student Grading System using ASP.NET MVC architecture, integrated with DeepInfra's NLP API for intelligent text analysis, grade prediction, and performance reporting. The system aims to reduce grading time, enhance consistency, minimize bias, and deliver actionable feedback to both students and instructors at the Institute of Southern Punjab, Multan.

1.3 Objectives

The primary objectives of this Automated Student Grading System are as follows:

- Develop a secure, role-based web platform using ASP.NET MVC architecture that supports Admin, Instructor, and Student functionalities.
- Implement secure login and authentication using token-based mechanisms to ensure authorized access to system resources.
- Enable course and assignment management where instructors can create, edit, and delete assignments, and students can view and submit their work.
- Integrate DeepInfra's NLP API to automatically analyze assignment submissions, evaluate grammar, content relevance, and structure, and predict grades.

- Generate detailed performance reports for instructors and admins, comparing predicted grades with actual grades, and highlighting risk levels and performance trends.
- Provide students with access to past results and feedback to help them identify weaknesses and improve their academic performance.
- Ensure database-backed storage and retrieval of user, course, assignment, grading, and performance history data for future reference.
- Minimize grading time and reduce bias by combining automated NLP-based evaluation with instructor review.

1.4 Scope

The scope of this Automated Student Grading System covers the design and implementation of a secure, role-based web platform using ASP.NET MVC, providing separate functionalities for Admin, Instructor, and Student users. The system includes course and assignment management, online assignment submission, automated grading through DeepInfra's NLP API, manual grading by instructors, detailed performance reporting, and database-backed storage of academic records. It focuses on improving grading speed, consistency, and feedback quality while enabling students to track their performance history. However, the project does not cover offline assignment handling, plagiarism detection, multimedia or non-text-based submission grading, or integration with external Learning Management Systems. All evaluation is limited to online, text-based submissions to ensure alignment with the project's objectives and available resources.

1.5 Significance

The Automated Student Grading System holds significant importance in modern education, where increasing class sizes and the demand for continuous assessment have made manual grading a time-consuming and often inconsistent process. By integrating advanced Natural Language Processing (NLP) through DeepInfra's API, the system enables fast, accurate, and unbiased evaluation of student assignments, reducing the workload on instructors and allowing them to focus more on teaching and student engagement.

A key feature of the system is that, after the automated grading process, instructors retain full control to manually adjust marks and provide personalized feedback before finalizing the student's result. This ensures that while the speed and consistency of AI-powered evaluation are utilized, the instructor's academic judgment and subject expertise remain central to the grading process.

The combination of automated predictions and manual verification delivers a balanced approach that maintains accuracy, fairness, and academic integrity.

This project contributes to the field of Educational Technology (EdTech) by demonstrating how AI-driven tools can be effectively applied in academic evaluation. It enhances fairness in grading, provides instant and detailed feedback to students, and facilitates long-term performance tracking, which is valuable for academic planning and intervention. For the Institute of Southern Punjab, Multan, this system not only streamlines academic workflows but also sets a foundation for adopting more AI-powered solutions in the future.

On a broader scale, the project showcases how scalable, secure, and intelligent grading systems can address global challenges in education, such as teacher shortages, assessment delays, and grading bias, thereby improving overall learning outcomes and institutional efficiency.

2. Literature Review

The literature review provides a comprehensive understanding of the existing body of knowledge in the domain of student grading and performance evaluation systems. It highlights the previous work carried out in this field, their strengths and weaknesses, and how the proposed Automated Student Grading System addresses the identified gaps. This section is divided into two parts: (2.1) Review of Related Work, and (2.2) Gap Analysis.

2.1 Review of Related Work

Traditionally, student assignments and examinations have been evaluated manually by instructors. While this approach allows for personalized assessment, it has several drawbacks. Manual grading is time-consuming, especially for large classes, and is prone to human bias and inconsistency. Furthermore, it delays feedback to students, which negatively impacts their ability to learn and improve. With the increasing student-to-teacher ratio in modern educational institutions, manual grading has become an unsustainable practice.

2.1.1 Previous System 1: Student Grade Prediction System

Overview:

The Student Grade Prediction System was primarily focused on estimating a student's academic performance through numerical indicators such as internal assessment marks, attendance records, and assignment scores. Instead of analyzing the quality of student submissions, the system relied on quantitative data to generate predictions.

The system provided a simple and user-friendly web interface where students could manually input their data. Based on predefined formulas or weightages assigned to each parameter (e.g., attendance = 10%, internal marks = 40%, assignments = 50%), the system calculated a performance score. This score was then mapped to a predicted grade, such as A, B, C, or D.

The main objective of this system was to allow students to gain an early understanding of their likely performance outcomes before final examinations. By simulating results in advance, students could identify weaknesses in their attendance or internal marks and take corrective actions during the semester.

However, while it helped in providing numerical predictions, the system lacked deeper analysis of academic work, as it could not evaluate the actual quality of assignments, comprehension, or writing skills.

Working Mechanism:

The system followed a straightforward process:

- Input Stage: The student entered internal marks, attendance records, and assignment marks.
- Processing Stage: The system applied predefined rules or formulas to calculate the overall performance score.
- Output Stage: The score was converted into a predicted grade (e.g., A, B, C, etc.), which was displayed to the student.

This approach made it simple for students to understand how attendance and internal marks contributed to their academic results.

Limitations:

Despite its usefulness, the system had significant limitations:

- It did not analyze the actual content of assignments (only numeric inputs).
- No integration of Natural Language Processing (NLP) to evaluate grammar, structure, or content relevance.
- Lack of automated feedback; students only received a predicted grade without guidance for improvement.
- No mechanism to identify students at academic risk.
- Instructors were not supported in reducing their manual grading workload.

As a result, while the system could predict grades, it did not contribute meaningfully to improving the learning process or ensuring fair evaluation.

Previous System 1 Architecture Diagrams:

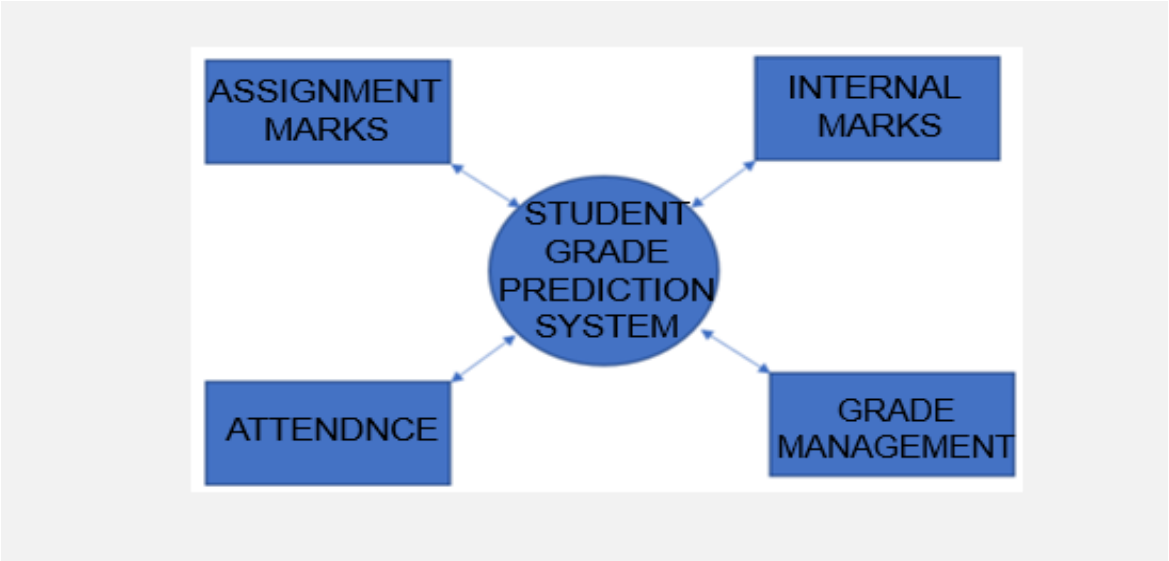


Figure 1: DFD of previous system 1

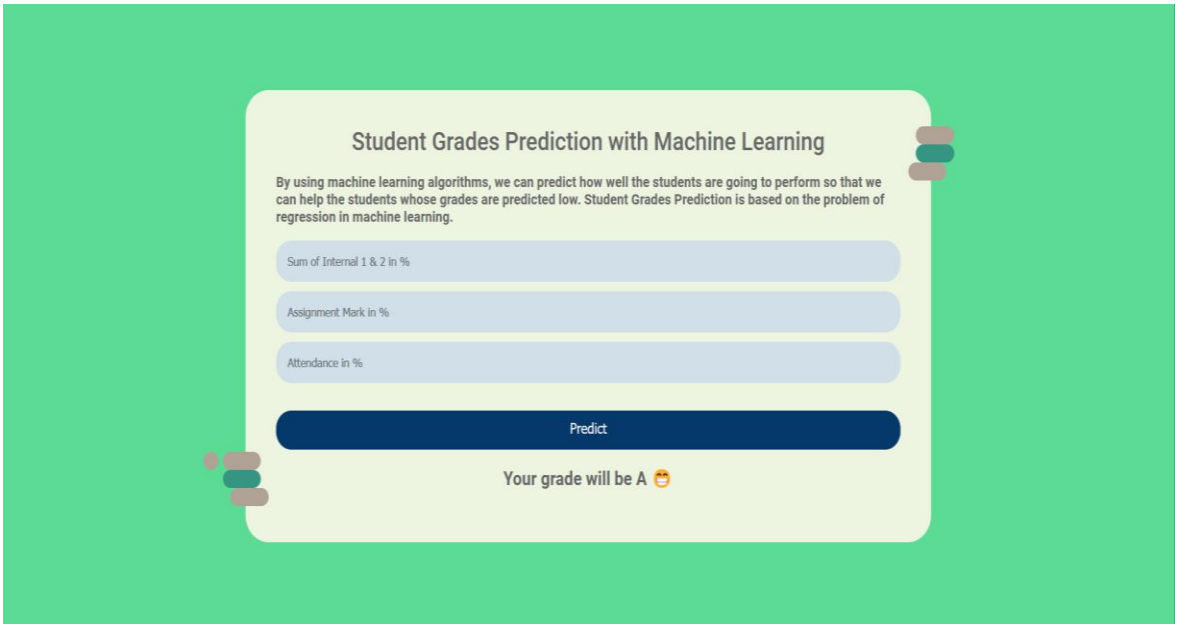


Figure 2: GUI Architecture of previous system 1

Conclusion:

The Previous System 1 which was Student Grade Prediction System successfully demonstrated a basic mechanism for forecasting student grades using numerical inputs such as attendance, internal assessments, and assignment scores. It provided a simple web-based interface that allowed students to anticipate their academic outcomes before final examinations. However, the system was limited in scope as it relied solely on numeric data and lacked advanced features such as Natural Language Processing (NLP), automated feedback generation, student risk-level analysis, and textual assignment evaluation. Consequently, while it offered a useful starting point for predictive analytics in education, it did not provide comprehensive support for instructors or students in terms of detailed performance insights, qualitative feedback, or continuous learning improvement.

2.1.2 Previous System 2: Student Grading System (SGS)

Overview:

The Previous System 2 which was Student Grading System (SGS) was a web-based application developed using the ASP.NET framework with a PostgreSQL database. It provided functionalities for instructors to create coursework such as assignments, exams, and projects, while students were able to upload their work according to instructor requirements (e.g., deadlines, file formats, file size). The system also enabled instructors to grade coursework based on predefined criteria and share results, scores, and comments with students. Additionally, instructors could manage their profiles, and students were able to view expectations, submitted work, and outcomes. It still relied heavily on manual grading by instructors and lacked advanced automation and intelligence features, such as NLP-based evaluation or predictive performance analysis.

Working Mechanism:

The Student Grading System (SGS) followed a structured workflow that primarily revolved around the interaction between instructors and students. The process can be explained in the following steps:

- **Coursework Creation:** Instructors logged into the system and created different types of coursework, such as assignments, projects, or examinations. Each coursework item was defined with specific requirements, including deadlines, file type restrictions, and file size limitations.
- **Submission of Work:** Students accessed the portal and uploaded their assignments or projects as per the instructor's requirements. The system validated submissions against

constraints (e.g., due date, file size, format).

- **Grading and Feedback:** Once submissions were received, instructors manually evaluated the work based on predefined grading rubrics and entered marks into the system. Feedback and comments were added for each submission, allowing students to see not just their scores but also the reasoning behind them.
- **Profile and Data Management:** Instructors could update their profiles within the system, while students could track their submissions, view graded coursework, and monitor outcomes. The PostgreSQL database stored all records of courses, submissions, grades, and feedback for secure access and future reference.
- **Results Access:** Students could log in and view the results of their submissions, including marks and instructor comments, providing them with clarity about their academic performance.

Limitations:

Despite its functionality, SGS had several limitations:

- **No NLP Integration:** The system lacked Natural Language Processing (NLP) for analyzing textual submissions.
- **No Automated Feedback:** Instructors had to manually evaluate and provide feedback, which was time-consuming.
- **No Grade Prediction:** The system did not include predictive analytics to estimate future student performance.
- **No Student Risk Identification:** It could not highlight at-risk students based on their performance trends.
- **Restricted to Manual Grading:** Fully dependent on instructors for evaluation, limiting efficiency in large-scale classrooms.

Previous System 2 Architecture Diagrams:

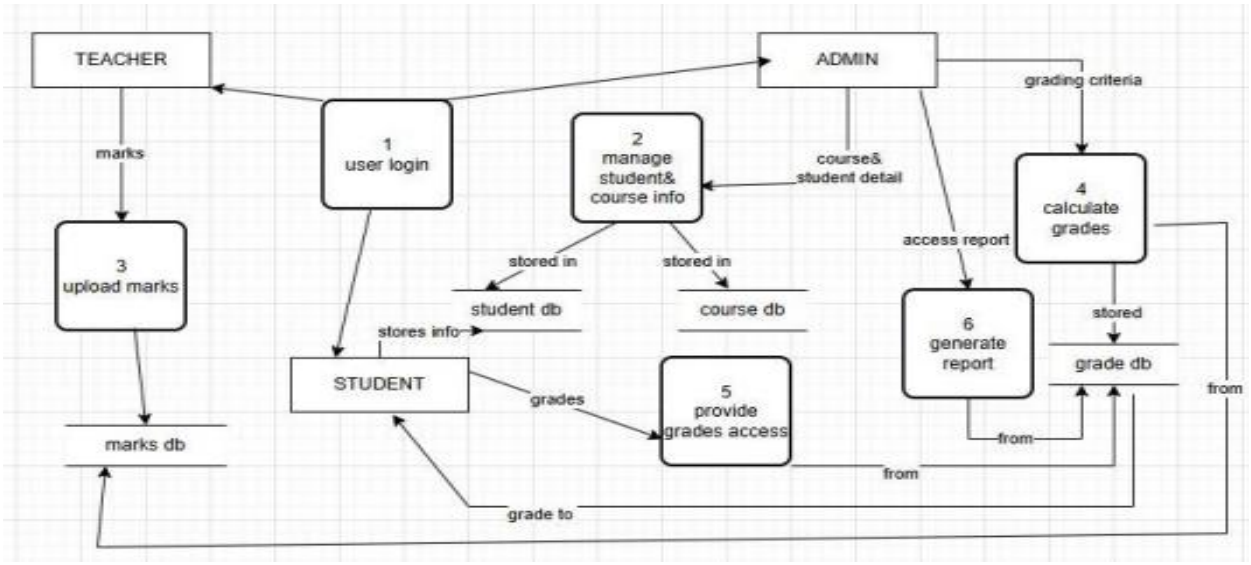


Figure 3: DFD of previous system 2

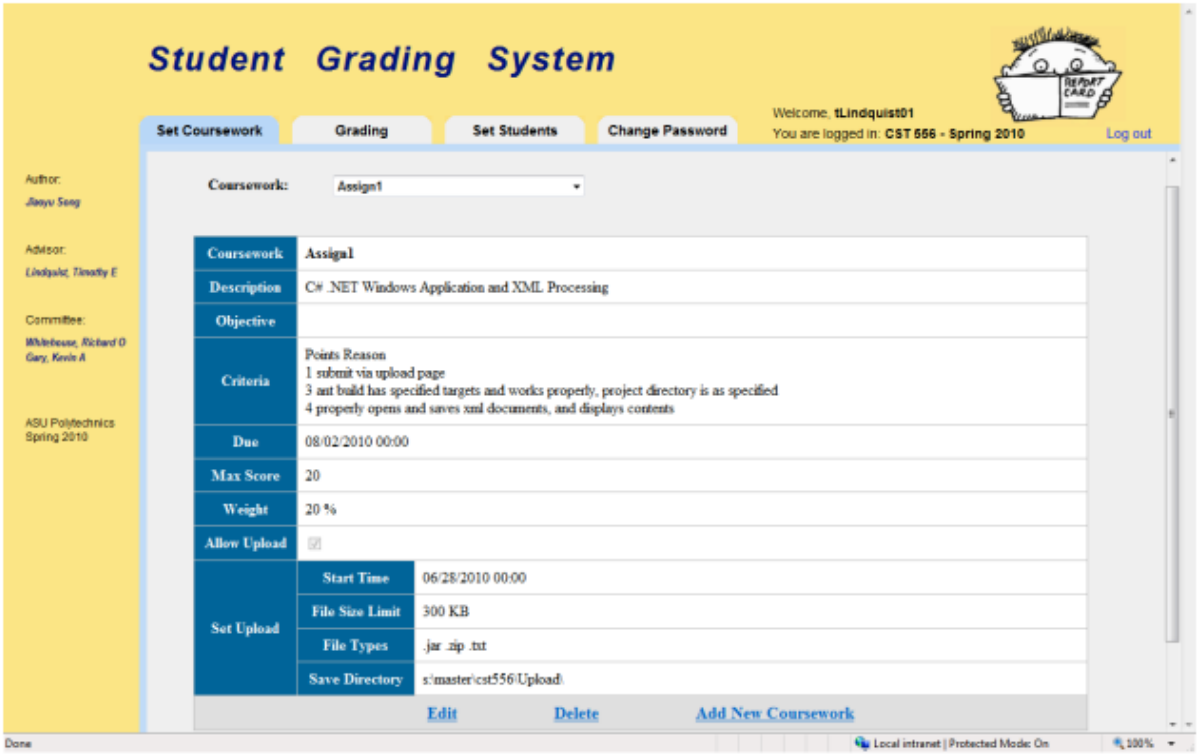


Figure 4: Instructor view of previous system 2

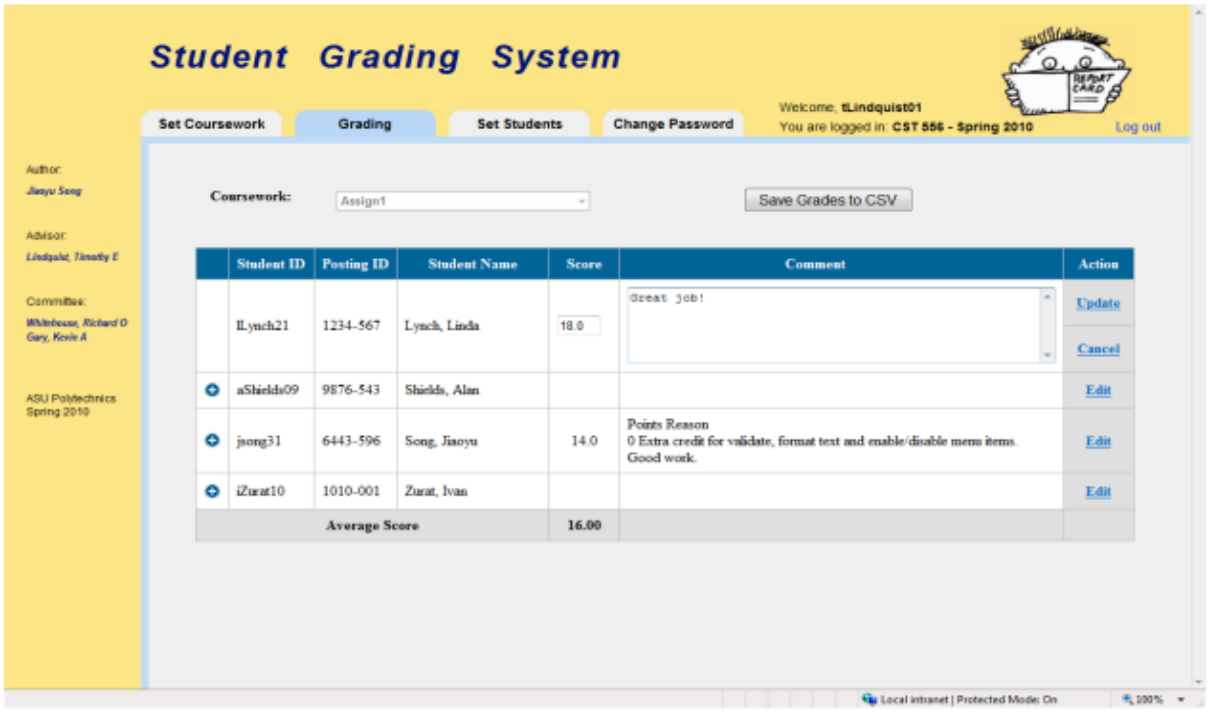


Figure 5: Student view of previous system 2

Conclusion:

The Student Grading System provided a structured way to manage assignments, grading, and feedback through a web platform, which was an improvement over basic grade prediction systems. However, it lacked advanced AI-driven features such as NLP-based content evaluation, automated grading, predictive performance analysis, and student risk-level monitoring. These limitations highlighted the need for a more intelligent, automated, and scalable system like the proposed Automated Student Grading System, which integrates NLP models to enhance grading efficiency and provide deeper insights for both students and instructors.

2.1.3 Summary of Limitations in Existing Systems

From the review of current and both previous systems, the following common limitations are identified:

- Lack of NLP-based analysis of student assignments.
- Absence of automated feedback generation to guide students in improving their work.
- No mechanism to predict student performance or risk levels for early intervention.
- Reliance on manual grading, which is time-consuming, inconsistent, and prone to bias.
- Limited or no reporting and analytics features to track long-term student progress.

- Inability to handle large-scale data efficiently, making them less scalable for universities.
- Lack of integration with AI/ML models, restricting deeper analysis of student submissions.
- Limited role-based access and security features, making them less flexible for multi-user environments.
- Absence of historical performance tracking, preventing students from analyzing their learning trends.
- No grade prediction or comparative reporting between automated and manual evaluations.
- Lack of student engagement tools such as progress summaries, personalized feedback, or alerts for weak performance.
- Systems mostly focused on numeric scores rather than holistic evaluation of content quality, grammar, and relevance.

2.2 Gap Analysis

The analysis of existing systems highlights several shortcomings that limit their effectiveness in providing comprehensive and intelligent grading solutions. While the previous systems addressed basic functionalities such as assignment submission, grade prediction, and manual evaluation, they lacked advanced mechanisms required to ensure fairness, consistency, and actionable feedback. Furthermore, there was no integration of intelligent NLP-based prediction, no automated feedback generation, and no distinct role-based user interfaces for Admin, Instructor, and Student. Considering these limitations, the following gaps have been identified:

2.2.1 Identified Gaps in Previous Systems

- **No NLP integration for text-based analysis:** Existing systems rely on numerical inputs such as marks, attendance, and assignment scores, or depend entirely on manual grading. None of them incorporate Natural Language Processing (NLP) models to evaluate the actual content, grammar, or relevance of student submissions.
- **No automatic feedback generation:** Students in the earlier systems could not receive meaningful, instant feedback based on their work. Feedback was either absent or limited to manual comments, reducing opportunities for timely improvement.
- **No risk level prediction for weak students:** Prior solutions did not include any mechanism for identifying students at academic risk. Without such predictive analysis, instructors lacked tools to provide early intervention or targeted support.

- **No combined approach of auto + manual grading:** Previous systems either focused only on manual grading or only on basic grade prediction. A hybrid model where automated grading works alongside instructor evaluation was missing, which could otherwise ensure both efficiency and fairness.
- **Limited reporting and no performance history:** Existing platforms lacked robust reporting tools and longitudinal tracking of student performance. This limited instructors' ability to monitor progress over time and students' ability to reflect on their learning journey.

2.2.2 Proposed Solution: Automated Student Grading System

The proposed Automated Student Grading System leverages modern Natural Language Processing (NLP) techniques through the integration of DeepInfra's external library. The system is designed to not only automate text-based grading but also combine it with traditional evaluation components to ensure a fair and comprehensive assessment process

i. **NLP-Based Grading and Prediction**

The use of DeepInfra's NLP API evaluates student submissions for grammar, content relevance, and structure. System predicts grades automatically and provides a comparison with manual grading to validate accuracy.

ii. **Automated Feedback Generation**

Generates instant and detailed feedback on every submission. Helps students understand strengths and weaknesses, improving self-learning without waiting for instructor remarks.

iii. **Risk Level Identification**

Identifies at-risk students through performance trends and predicted grades. Allows instructors to take early corrective action (extra guidance, remedial work, counseling).

iv. **Hybrid Evaluation (Auto + Manual Grading)**

Combines automatic NLP-based grading with manual instructor grading. Ensures fairness, flexibility, and consistency in evaluation.

v. **Comprehensive Reporting and Performance Tracking**

Generates detailed reports (actual vs predicted grades, subject-wise performance). Students can view performance history to track improvement over time.

vi. **Role-Based User Interfaces**

Provides separate dashboards for Admin, Instructor, and Student. Ensures secure, role-

based access and a user-friendly environment missing in earlier systems.

vii. **Course and Assignment Management**

Instructors can create/manage courses, assignments, quizzes, and exams. Students can upload submissions securely within deadlines.

viii. **Final Grade and GPA Calculation**

System integrates assignments, quizzes, mid-term, final-term, and attendance into a final grade. Produces both letter grade and numerical GPA automatically.

Conclusion

The Automated Student Grading System successfully bridges the shortcomings identified in previous systems by integrating Natural Language Processing (NLP) for automated grading and prediction, generating instant feedback, and providing risk identification for weaker students. Unlike earlier platforms, it combines both automatic and manual grading to ensure fairness and flexibility while offering role-based user interfaces for Admin, Instructor, and Student. Furthermore, the inclusion of comprehensive reporting and performance tracking makes it a complete academic evaluation tool. By filling these gaps, the proposed solution not only enhances grading efficiency and consistency but also contributes to improved student learning and informed decision-making by instructors and administrators.

2.2.3 Comparison Table

Table 1: Comparison of Previous Systems and Proposed System

Feature / Criteria	Previous System 1	Previous System 2	Proposed Automated Student Grading System
Technology Used	Web-based page, simple input	ASP.NET + PostgreSQL	ASP.NET MVC + SQL Server + DeepInfra NLP
User Roles	Single (Student input only)	Instructor, Student	Admin, Instructor, Student (Role-based)

Assignment Submission	Not available	Available (file upload, due date, file size/type check)	Available (online, text-based, secure submission)
Grading Type	Predicted grades only	Manual grading	Auto grading via NLP + Manual grading (hybrid)
NLP Integration	No	No	Yes (DeepInfra API for grammar, relevance, prediction)
Automatic Feedback Generation	No	Limited comments only	Yes (detailed feedback + performance summary)
Grade Prediction	Yes (basic, using attendance & internals)	No	Yes (NLP-based prediction + comparison with actual)
Risk Level Prediction (Weak Students)	No	No	Yes (at-risk students flagged)
Reporting & Performance History	No	Limited outcomes view	Comprehensive reports (grades, predicted vs actual, trends)
Instructor Analytics (Graphs/Charts)	No	No	Yes (Graphical view of student grades & progress)
User Interface	Basic web page	Simple ASP.NET forms	Role-based dashboards (modern MVC views)
Flexibility & Scalabilit	Very limited	Moderate	High (modular MVC design, scalable DB)

3. Methodology

This chapter explains the approach, tools, and techniques that were followed during the development of the Automated Student Grading System. It describes the system design process, selection of technologies, implementation strategies, and testing mechanisms adopted to ensure that the system met its objectives of accuracy, scalability, and usability.

3.1 Project Design

3.1.1 System Architecture

The Automated Student Grading System was designed using the Model-View-Controller (MVC) architecture provided by ASP.NET Core. This design pattern separates the application into three layers:

Models: Represent the academic entities (Students, Teachers, Courses, Assignments, Results).

Views: Contain the user interfaces for Admin, Instructor, and Student.

Controllers: Act as the bridge between the front-end and back-end, processing user requests, executing business logic, and interacting with the database.

This separation of concerns ensured better maintainability, scalability, and reusability of the system's code.

3.1.2 Workflow Diagram

The system workflow proceeds as follows:

i. **Authentication & Role-Based Login:**
   - Admin, Instructor, and Student each have separate login credentials with role-specific dashboards.
   - Admin manages system users, roles, and overall monitoring.
   - Instructors create assignments, quizzes, and manage results.
   - Students submit assignments and view results/feedback.

ii. **Student Workflow:**
   - Student logs into the portal and uploads an assignment in .docx format.
   - The system validates the submission and securely stores it in the database.
   - After grading, the student can view assignments, feedback, attendance, quizzes, mid-term, and final-term results in their dashboard.

iii. Grading Process:

- The uploaded assignment text is analyzed by two grading engines:
- DeepInfra API (AI-powered evaluation) for grammar, structure, and content relevance.
- NLP Service (rule-based evaluation using ML.NET) for keyword coverage, similarity, and structure validation.
- Both engines generate numeric scores and textual feedback.

iv. Instructor Workflow:

- Instructor receives the grading results in their dashboard.
- Instructor can review, adjust, or override AI predictions for fairness.
- Instructor can evaluate complete student performance based on:
  - Assignments
  - Quizzes
  - Attendance
  - Mid-term and Final-term exams
  - Instructor can also provide additional manual feedback to students.

v. Result Finalization:

- Once reviewed and approved by the instructor, results are finalized.
- Final grades and feedback are published for students to access.

vi. Data Management:

- All user interactions, submissions, and results are securely stored in SQL Server.
- The system ensures traceability and transparency of the grading process.

3.1.3 Design Considerations

Fairness

One of the biggest challenges in automated grading is bias. If grading is done purely by AI, it may favor certain writing styles or miss context. To avoid this, the system integrates two grading engines:

- **AI-based (DeepInfra API):** Evaluates grammar, clarity, and relevance.
- **Rule-based (NLP Service):** Uses similarity checks, keyword matching, and structure validation.

By combining both, the system ensures more balanced, fair, and transparent grading, reducing the risk of biased results.

Security

- Since the system stores sensitive academic data such as student identities, assignments, and grades, ensuring security is crucial. The design includes:
- Authentication mechanisms (username/password login).
- Role-based access control (Student, Instructor, Admin) so that users can only access features relevant to their role.
- Database protection with secure queries to prevent unauthorized access.
- This guarantees that student records remain confidential and tamper-proof.

Scalability

The system is designed to handle a large number of simultaneous submissions (e.g., during exam periods). To achieve this:

- SQL Server is optimized with efficient queries.
- Controllers and services use asynchronous processing to avoid blocking.

The system can be deployed on Microsoft Azure cloud, making it capable of scaling horizontally when needed. This ensures that even if thousands of assignments are submitted, the system will remain stable and responsive.

Usability

A grading system is only successful if students and instructors can use it easily. Therefore, the system provides role-specific dashboards:

- Students: Simple interface to submit assignments and view results.
- Instructors: Tools to create assignments, review AI-predicted grades, and adjust results.
- Admin: Management of users, roles, and system settings.

The design focuses on clean navigation, minimal steps, and clarity of feedback, ensuring a positive user experience.

3.1.4 Data Flow

Input: Students upload .docx assignments.

Processing:

- DeepInfra API evaluates grammar, clarity, and relevance.
- NLP Service applies similarity checks, keyword coverage, and structure validation.

Storage: Scores and feedback are stored in SQL Server.

Output: Instructors review results; students view approved grades and feedback.

3.2 Technology Stack

3.2.1 Programming Languages

- Backend: C# with ASP.NET Core MVC
- Frontend: HTML, CSS, JavaScript, Bootstrap

3.2.2 Frameworks & Libraries

- ASP.NET Core MVC for application structure
- Entity Framework Core for ORM and database handling
- DeepInfra API for AI-powered grading
- ML.NET + OpenXML SDK for NLP-based evaluation

3.2.3 Database

- Microsoft SQL Server for managing users, assignments, and results

3.2.4 Tools & Platforms

- Visual Studio 2022 (development)
- GitHub (version control)
- IIS for local deployment

3.2.5 Hardware/Cloud Requirements

- Tested on Intel i5, 8 GB RAM, 256 GB SSD
- Optional cloud deployment via Microsoft Azure

3.3 Implementation

3.3.1 Data Collection & Preprocessing

In the proposed system, students submit their assignments in the form of .docx documents through the web portal. Once uploaded, the system automatically extracts the raw text content from these files. Any unnecessary formatting, such as font styles, images, extra spaces, or non-textual elements, is removed to ensure a clean and standardized text input. This preprocessing step is crucial because it eliminates formatting noise that could otherwise interfere with the grading logic. After cleaning, the text is further normalized such as converting to lowercase, removing special symbols, and ensuring consistent sentence structures so that the input remains uniform across all submissions. This standardized version of the student's response is then passed to the grading

engine, which integrates both rule-based checks and the DeepInfra NLP API for advanced evaluation.

By applying these preprocessing steps before analysis, the system guarantees that all assignments are evaluated on a fair and equal basis, unaffected by formatting differences, and that the NLP model can perform more accurate grading based purely on content quality, relevance, and clarity.

3.3.2 Model Development

The grading mechanism of the Automated Student Grading System was designed as a hybrid evaluation framework, ensuring both fairness and accuracy. Instead of relying solely on AI-based predictions, the system integrates two complementary approaches:

(a) AI-Driven Evaluation (DeepInfra)

This module leverages advanced Large Language Models (LLMs) such as LLaMA and Mixtral through DeepInfra's API. When a student submits an assignment, the system generates a grading prompt containing the question, model answer, and evaluation criteria (relevance, accuracy, completeness, and clarity). The LLM processes this input and outputs a numeric score between 0 and 10, reflecting the overall quality of the student's response. This approach ensures a context-aware and language-sensitive assessment, capable of evaluating grammar, coherence, and depth of understanding.

(b) Rule-Based Evaluation (NLP Service)

To complement the AI-driven scoring, the system also employs a rule-based grading engine built using ML.NET and NLP techniques. This service measures similarity between the student's answer and the model solution through methods such as TF-IDF vectorization, keyword matching, and structural analysis. In addition to producing a numerical score, this module also generates detailed feedback, highlighting missing keywords, incomplete points, or structural issues. The rule-based approach ensures transparency and provides explainable grading that AI-only models cannot always guarantee.

By combining these two evaluation strategies, the system achieves a balance between human-like intelligence (AI-driven scoring) and objective, explainable assessment (rule-based scoring), resulting in more reliable and fair grading outcomes.

3.3.3 System Integration

- **Student Module:** Allows submission of assignments and retrieval of results.
- **Instructor Module:** Provides result review, manual adjustments, and finalization.

- **Admin Module:** Manages users and system settings.
- **Grading Engines:** DeepInfra API and NLP Service run in parallel, ensuring hybrid evaluation.

3.3.4 User Interface Development

The system was designed with three distinct dashboards, each tailored to the needs of its specific user role:

1. Student Dashboard

- Allows students to submit assignments in text format.
- Provides access to graded results generated by the AI and instructors.
- Displays feedback and suggestions for improvement, helping students learn from their mistakes.

2. Instructor Dashboard

- Enables instructors to create and assign new assignments.
- Allows reviewing of AI-predicted grades generated by the DeepInfra engine.
- Provides functionality to adjust grades manually where necessary, ensuring fairness.
- Supports adding qualitative feedback on each submission, giving students detailed insights into their performance.

Facilitates generation of final results, combining AI grading and instructor evaluation.

3. Admin Dashboard

- Used to manage users, roles, and system permissions.
- Provides tools for monitoring overall system performance and ensuring data integrity.
- Allows administrators to configure system-wide settings, including integration with APIs and security management.

3.4 Development Model (Spiral Model)

Explanation:

The Spiral Model was selected as the development methodology for this Automated Student Grading System because it combines the structured nature of the Waterfall model with the flexibility of Iterative development. Unlike linear models, the Spiral Model emphasizes risk management, continuous feedback, and incremental improvements, making it highly effective for systems that require reliability and adaptability.

Working of Spiral Model:

The model progresses in cycles (spirals), and each cycle involves four stages:

Planning: In this project, the requirements of students (assignment submission, result viewing), instructors (assignment creation, feedback, grade adjustment), and administrators (user and role management) were defined.

Risk Analysis: Potential challenges such as accuracy of AI grading, fairness of automated results, and system scalability were evaluated. Backup strategies like instructor overrides and optimized database queries were designed to handle these risks.

Development & Implementation: The core functionalities were developed in phases. In the first cycle, assignment submission and storage were implemented. In the next cycles, DeepInfra AI-based grading, instructor adjustments, and dashboards were integrated gradually.

Testing & Evaluation: Each prototype was tested with real-world scenarios. Students and instructors provided feedback, which was incorporated into the next development cycle to ensure usability and fairness.

Why Spiral Model was used in this Project:

Risk Management: Since grading fairness and AI reliability were critical concerns, Spiral Model allowed risk analysis at every stage.

Flexibility: The system required multiple roles (student, instructor, admin) and gradual integration of AI services. The iterative nature of Spiral Model made this integration smoother.

Continuous Feedback: Instructor and student feedback was incorporated after each testing phase, improving the quality of the final product.

Scalability: The model allowed the project to start small (basic submission + grading) and expand later with advanced features like feedback, overrides, and dashboards.

3.5 Testing

3.5.1 Testing Strategy

It describes the overall plan: unit testing of modules (login, upload, grading), integration testing between controllers, services, and DB, and full system/end-to-end testing across all user roles.

3.5.2 Test Cases

Covers typical and edge scenarios: correct answers (full marks), partial answers (proportional grading), irrelevant/empty submissions (zero marks), invalid file uploads, and multiple students

uploading simultaneously for scalability.

3.5.3 Performance & Security Testing

Evaluates efficiency under load (100+ submissions in under one minute), response times, database query optimization, as well as security checks like file upload validation, role-based access, and protection against SQL injection, CSRF, and XSS.

3.5.4 User Acceptance Testing (UAT)

Summarizes feedback from instructors and students: instructors validated fairness and override functionality, while students confirmed usability, timely feedback, and transparency.

3.5.5 Results & Analysis

The evaluation showed a high degree of alignment between manual grading and the automated grading engine. Manual grading by instructors was taken as the benchmark for accuracy, while the automated system utilized a hybrid approach combining DeepInfra's LLM-based evaluation with rule-based checks (keyword relevance, clarity, and structural coverage).

The comparison revealed that:

- The automated system achieved around 90% accuracy when compared to manual grading, ensuring fairness and reliability.
- DeepInfra's language model provided contextual understanding and semantic evaluation, while the rule-based layer ensured grading consistency under the predefined conditions.
- In scenarios where answers were subjective or slightly incomplete, the hybrid approach maintained balance by giving proportional scores rather than extremes.
- Overall, the automated system proved to be efficient, transparent, and scalable, producing results in real-time while staying closely aligned with human evaluators.

This demonstrates that the system not only reduces grading time significantly but also delivers results that are trustworthy and consistent with the grading criteria set by instructors.

4. System Requirements

4.1 Functional Requirements

Functional requirements describe the features and behaviors that the Automated Student Grading System must perform. These are the services provided to different types of users:

1. User Registration and Role Assignment:

The admin should be able to create new user accounts and assign them a role (Student, Instructor). Each user will be uniquely identified by a User ID.

2. Secure Login and Authentication:

All users must log in using their credentials (username and password). The system will verify users through a secure token-based authentication mechanism.

3. Course Management (Admin)

In this Automated Student Grading System, the **Admin** holds the primary responsibility for course management. The Admin creates and manages all courses within the system, where each course is defined by a unique Course ID, course name, semester, and department details. Once a course is created, the Admin assigns it to a specific instructor, ensuring that every course has a designated faculty member. Additionally, the Admin enrolls students into courses according to their semester and department. Instructors, on the other hand, do not create courses but are able to view and manage only those courses allotted to them by the Admin. They can access the list of enrolled students and monitor their submissions within the assigned courses.

4. Assignment Management (Instructor)

Instructors can create, edit, and delete assignments within their courses. Assignments include a title, description, due date, and associated course.

5. Assignment Submission (Student)

Students can view available assignments in their enrolled courses. Students can upload and submit their work before the deadline. Each submission includes the submission date, file path/text, and is linked to a student and assignment.

6. Grading (Instructor)

Instructor can view student submissions and use the built-in NLP model for automated grading of assignments. Alongside auto-grading, instructors can manually enter marks for quizzes, attendance, midterm, and final exams. The system then compiles all components to generate the

student's final grade, with grades and feedback stored securely against each student and course.

7. NLP-based Analysis

When a student submits an assignment, the instructor can perform auto-grading using NLP techniques. The system analyzes the submission, predicts a grade, and generates feedback. All results are stored in the database for later review, reporting, and final grade calculation.

8. Performance Monitoring & Reporting

Instructors can view a graphical dashboard of student performance, including predicted grades. The system also highlights students who are at risk, making it easier for instructors to take timely action. Admins can generate and view overall result reports for record-keeping and monitoring.

9. Feedback for At-Risk Students

Instructors can provide feedback to students identified as at risk after grading. Students can view this feedback through notifications on their dashboard and respond back to the instructor. Both students and instructors can track the feedback history directly from their dashboards for continuous improvement.

4.2 Non-Functional Requirements

1. Security

The system must encrypt passwords and authentication tokens. User sessions must be protected using secure HTTPS protocols. Role-based access control must be implemented to restrict access to sensitive features and data.

2. Usability

The interface must be intuitive and user-friendly for all types of users. Users should be able to navigate the platform with minimal training or instruction.

3. Availability

The system should be available 24/7 except during scheduled maintenance. The target system uptime is 99.5% or higher.

4. Performance

Page load and system response times must be under 3 seconds for most operations. NLP analysis should complete within 5 seconds for an average-length submission.

5. Scalability

The system should be able to scale horizontally to handle an increasing number of users,

assignments, and submissions. The database should support thousands of concurrent entries and queries without performance degradation.

6. Compatibility

The system must work across major modern web browsers (Chrome, Firefox, Edge). It should support access from desktops, laptops, tablets, and smartphones.

7. Maintainability

The system code base should follow best practices and be modular to support future updates or feature additions. Logs should be maintained for user activity and system events to aid troubleshooting.

8. Backup and Recovery

Regular data backups should be scheduled automatically. In case of failure, the system should support data recovery with minimal loss.

5. System Design

5.1 Design Overview

The Automated Student Grading System has been designed as a web-based platform following the ASP.NET Core MVC (Model–View–Controller) architecture. The system is structured around three primary user roles Students, Instructors, and Administrators, each interacting with the system through dedicated modules.

At a high level, the system is composed of the following major components:

1. User Management Module

This module is responsible for secure authentication, authorization, and role assignment within the system. It ensures that only registered and verified users can log in and access features based on their role (Student, Instructor, or Admin). By implementing robust login mechanisms, it prevents unauthorized access and protects sensitive academic data. Role-based access control guarantees that students can only perform academic activities, instructors can manage courses and grading, and administrators can handle system-wide configurations.

2. Assignment Management Module

The assignment management module handles the creation, submission, and tracking of assignments. Instructors can design new assignments, set deadlines, and monitor student submissions. Students, on the other hand, can upload their answers in text format through an intuitive interface. The system validates each submission and ensures data consistency. This module forms the foundation of the grading pipeline since it supplies raw student input to the grading engine for evaluation.

3. Automated Grading Engine

At the core of the system lies the automated grading engine, which performs both AI-powered and rule-based evaluations. It integrates DeepInfra's NLP API to assess grammar, relevance, and structural coherence using advanced large language models. Alongside AI evaluation, rule-based checks such as keyword matching, structural validation, and similarity scoring are applied to ensure a fair and balanced assessment. The engine generates preliminary grades and detailed feedback, drastically reducing the time and effort required compared to manual grading.

4. Instructor Review Module

This module provides instructors with oversight over the automated grading process. Once the

grading engine produces results, instructors can review, validate, or override grades to ensure fairness and transparency. Instructors may also append personalized feedback to guide student improvement. This dual-layer approach (AI grading + instructor review) balances efficiency with human judgment, ensuring that the system remains both reliable and academically sound.

5. Results and Feedback Module

The results and feedback module manages storage and distribution of final grades. Once approved by instructors, graded results are securely stored in the SQL Server database and made accessible to students. Each student can log in to view grades, performance analysis, and detailed feedback. The system also maintains historical performance data, allowing students and instructors to track academic progress over time. This ensures transparency, accountability, and continuous improvement.

6. Administration and Monitoring Module

The administration module ensures the overall stability, security, and performance of the platform. Administrators manage user accounts, system configurations, and role assignments. They also monitor system performance, resolve technical issues, and optimize resources. By enforcing security protocols and maintaining data integrity, this module guarantees that the system operates smoothly and efficiently, even under heavy usage.

5.1.1 Use Case Diagrams

Use Case 1

Actors: All Users (Admin, Teacher, Student)

The Figure 6 shows the interactions between different actors and the system for managing user accounts. The main actors include:

- Admin: Manages the overall system and has full access rights.
- Instructor: Can access login roles after authentication to perform teaching-related tasks.
- Student: Can access login roles after authentication to perform student-related tasks.

User (General): Represents anyone interacting with the system, including Admin, Instructor, and Student.

The main use cases are:

- Create Account: Allows a new user to register in the system.
- Authenticate User: Verifies user credentials before granting access.
- Access Login & Role: Provides access based on the user's role after successful

authentication.

The "Authenticate User" use case is included in the "Access Login & Role" use case, ensuring that only authenticated users can proceed

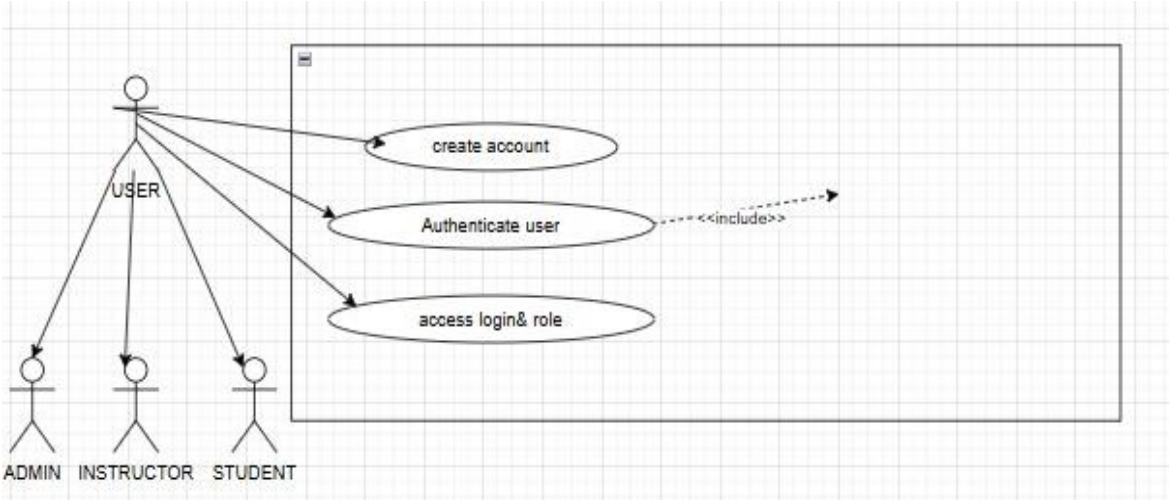


Figure 6: Use Case Diagram for User Account Management System

Use Case 2

Actor: Admin

In this use case, the **Admin** plays a central role in managing the overall system. The admin is responsible for handling the organizational structure by managing departments and semester-wise subjects. Additionally, the admin manages user accounts, which may include both students and instructors, ensuring smooth access to the system. Apart from account management, the admin also has the ability to view results and maintain data integrity by saving important information. Various operations such as adding, editing, and deleting data are included in the admin's responsibilities, making this role essential for the effective functioning of the system.

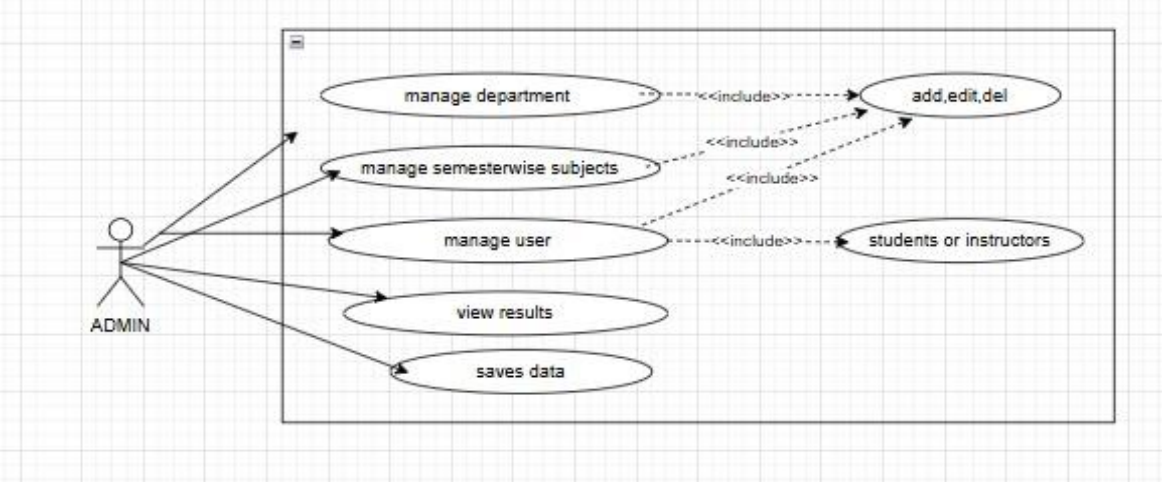


Figure 7: Admin - Use Case Diagram

Use Case 3

Actor: Teacher

Description:

The **Teacher** interacts with the system both as an instructor and an evaluator. Teachers manage their profile and update student information when needed. They can upload documents such as assignments or lecture materials for student access. Teachers also evaluate students by uploading results manually or using natural language processing (NLP) for grading. They then save results in the system and generate feedback for students, helping them to monitor and improve performance.

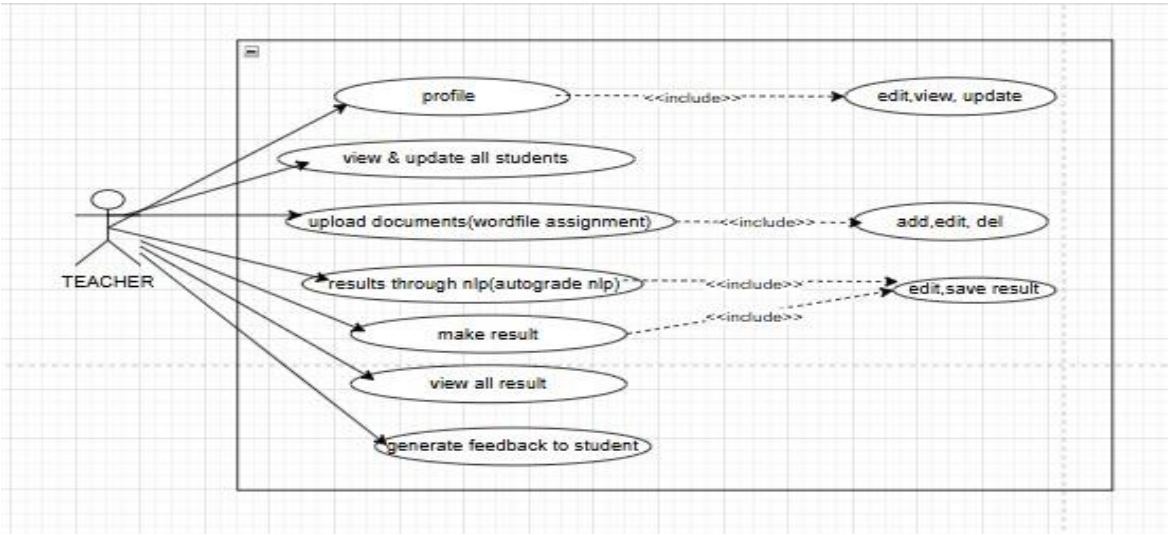


Figure 8: Teacher – Use Case Diagram

Use Case 4

Actor: Student

Description:

The **Student** primarily uses the system for academic purposes. Students can create and update their profile, upload assignments, and check notifications. They are able to view their results and read feedback shared by teachers. The system also generates automated responses to student activities, making their learning experience interactive. Thus, the student represents the main learning actor within the system.

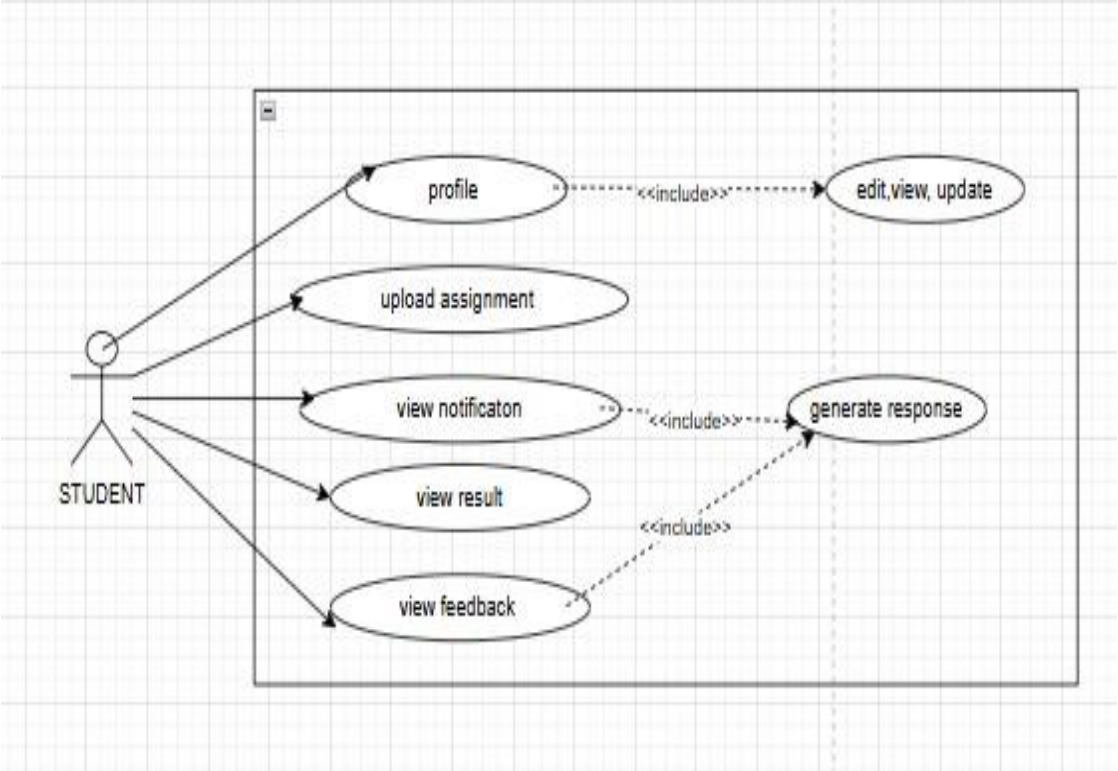


Figure 9: Student – Use Case Diagram

Use Case 5

Actors: Admin, Teacher, Student, User

Description:

The overall system integrates all four actors. The Admin manages departments, subjects, users, and results. The Teacher creates content, evaluates students, and provides feedback. The Student submits assignments, views notifications, and checks results. The User represents any individual accessing the system, with functions like account creation, authentication, and login. Together, these actors show the complete workflow of the Automated Student Grading System.

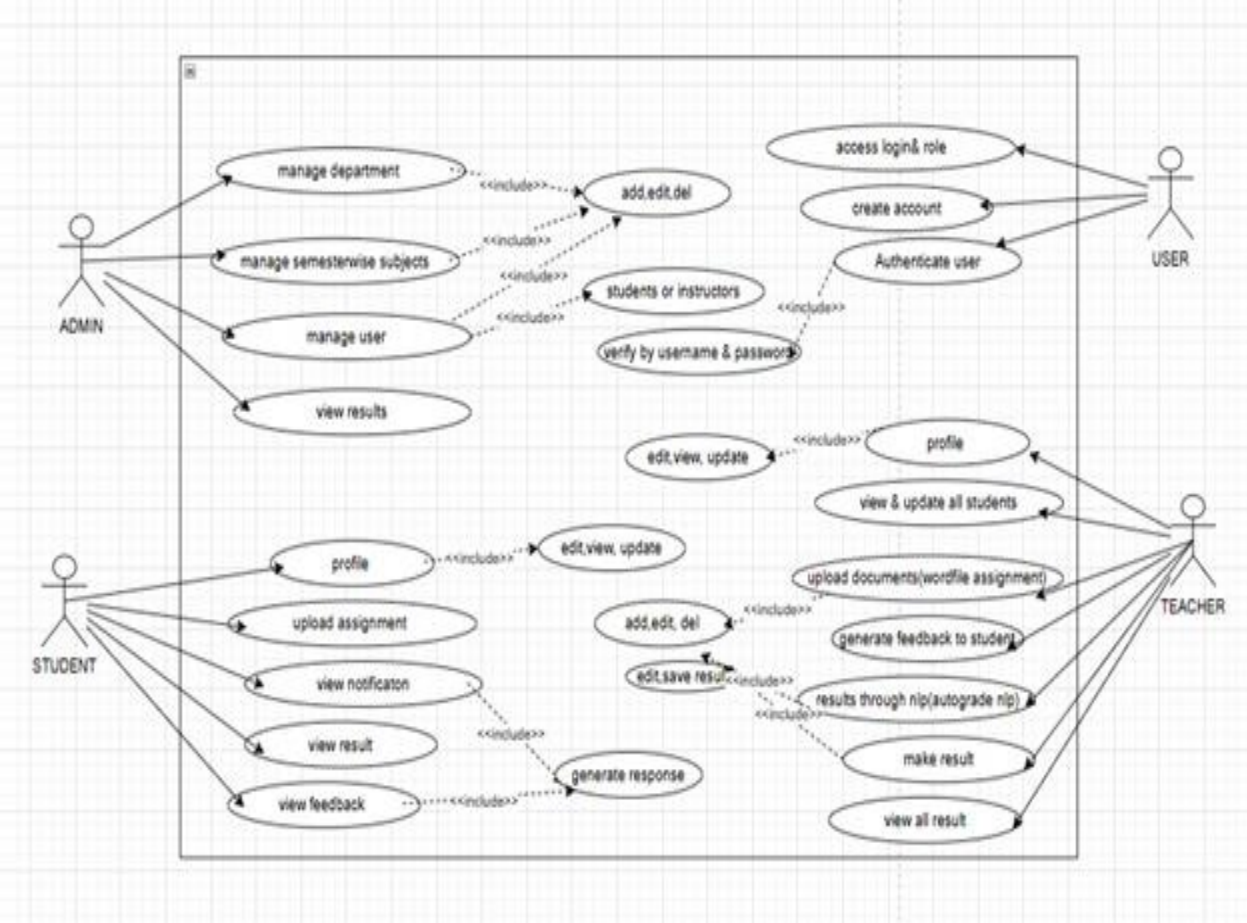


Figure 10: Automated Student Grading System – Use Case Diagram

5.2 Architecture

The architecture of the proposed system defines how different modules and components interact with each other to meet the functional requirements of student performance prediction and auto-grading. A System Architecture Diagram is included to visually represent the layered design of the system. This diagram highlights the major components such as the Users Layer, User Interface (Frontend), Application Logic Layer, Database Layer, Visual Dashboards, Security & Access Control, and Integration Layer, along with the interactions between them. It gives a high-level view of how users communicate with the system, how data is processed and stored, and how security and visualization are maintained.

Since the system is primarily data-driven, its architecture is further represented using Data Flow Diagrams (DFDs). These diagrams provide a clear picture of how data moves across various layers of the system. They demonstrate the flow of information between processes, external entities, and

data stores, giving a simplified yet accurate understanding of the system's internal structure and functionality.

5.2.1 System Architecture Diagram

In this project, the overall system architecture is represented through The architecture of the Student Performance Prediction and Auto-Grading System follows a layered design to ensure modularity, scalability, security, and maintainability.

1. Users Layer

Actors: Admin, Instructor, Student.

Access: Each role is provided with a customized dashboard.

Responsibilities:

- Admin: System management, user control, monitoring
- Instructor: Upload assignments, manage grading, monitor student performance.
- Student: Submit assignments, view results, track performance.

2. User Interface (Frontend)

- Provides role-based dashboards.
- Enables interaction & visualization (assignments, grades, predicted results).
- Acts as a bridge between users and application logic.

3. Application Logic Layer

Core layer for managing business rules and workflows.

Key functionalities:

Auto-grading of assignments.

- NLP-based performance prediction (predict student results from submitted documents, quizzes, and past records).
- Manages communication between frontend and database.

4. Database Layer (SQL Server 2008+)

Stores all structured system data:

- Users, Roles, Departments, Semesters, Subjects.
- Assignments, Uploaded Documents, Feedback.
- Predicted & Actual Student Results.

Ensures reliable, consistent, and secure data storage and Supports relationships between entities via foreign keys.

5. Visual Dashboards

Provides real-time analytics and reporting.

Role-based insights:

- Students: Personal grades, progress trends.
- Instructors: Class performance, grading summaries.
- Admins: Department-wide monitoring, system usage analytics.

Generates charts, graphs, and comparative performance analysis.

6. Security & Access Control

- Implements role-based access control (RBAC).
- Provides authentication & authorization for all users
- Ensures data confidentiality, integrity, and restricted access.
- Protects sensitive student and institutional data.

7. Integration Layer

- Designed for future integration with the official university portal.
- Ensures expandability for third-party systems (e.g., LMS, ERP, or cloud-based services).
- Facilitates data exchange with external academic platforms.

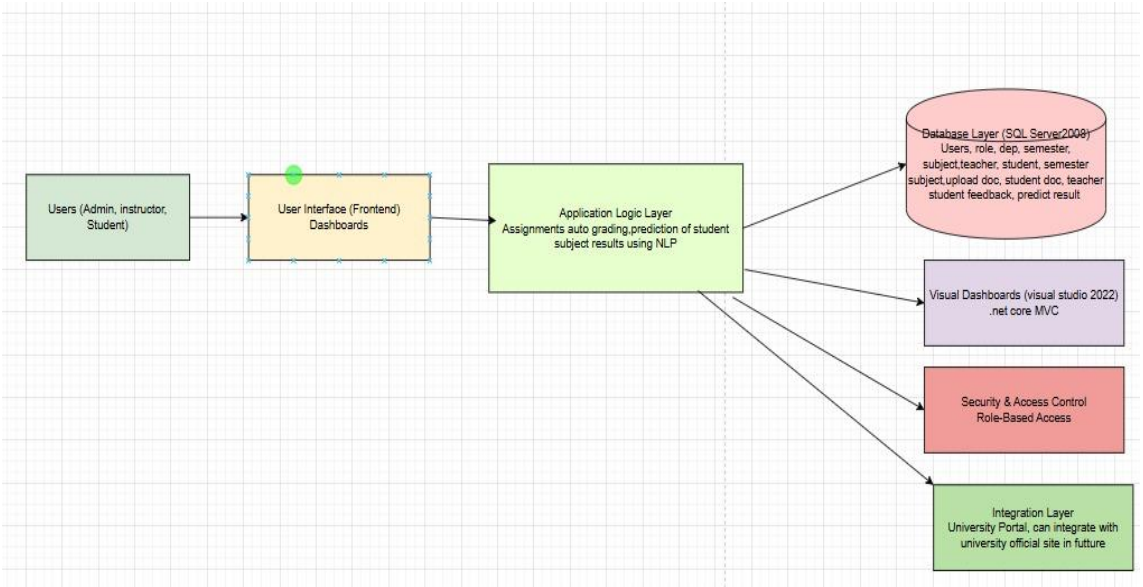


Figure 11: System Architecture Diagram

5.2.2 Data Flow Diagrams (DFDs)

DFD Level 0

The Level 0 DFD provides a high-level overview of the Automated Student Grading System. It shows the system as a single process and highlights how it interacts with external entities:

- Student: Students create accounts, submit assignments, view assignment details, receive feedback, and check grades.
- Instructor: Instructors upload assignments, evaluate submissions, view reports, and provide feedback to students. They also interact with NLP tools for grading and plagiarism detection.
- Admin: The admin manages account creation, user roles, and generates reports.

At this level, the system itself is represented as one main process ("Automated Student Grading System") that receives inputs (e.g., submissions, login requests) and produces outputs (e.g., grades, reports, feedback).

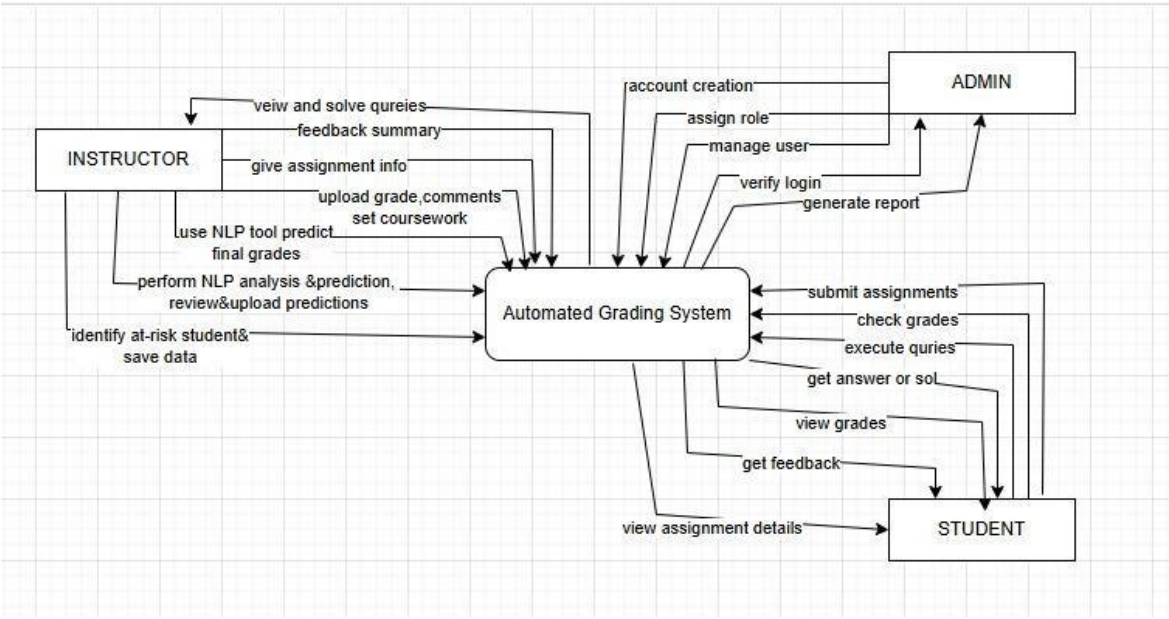


Figure 12: (DFD) Level 0 of the Automated Student Grading System

DFD Level 1

This DFD Level 1 provides a detailed view of how the Automated Student Grading System works

internally. The system is divided into seven main processes:

- Authentication: Validates user credentials when students, instructors, or admins log in.
- User & Role Management: Allows the admin to create accounts, assign roles, and manage users.
- Assignment Management: Instructors upload and manage coursework, while students submit assignments through this process.
- Submission Data Storage: Stores assignment data securely for grading and further processing.
- Plagiarism & Similarity Check: Uses NLP and automated tools to check for copied or similar content in submissions.
- Report & Grade Analysis: Generates grading reports and statistical analysis for instructors and admins.
- Feedback & Result Generation: Prepares detailed feedback for students and final grades based on evaluation.

This detailed DFD shows how data flows through multiple processes from student submission to plagiarism detection, grading, report generation, and final feedback delivery. It also demonstrates how admins and instructors interact with the system for management and oversight purposes.

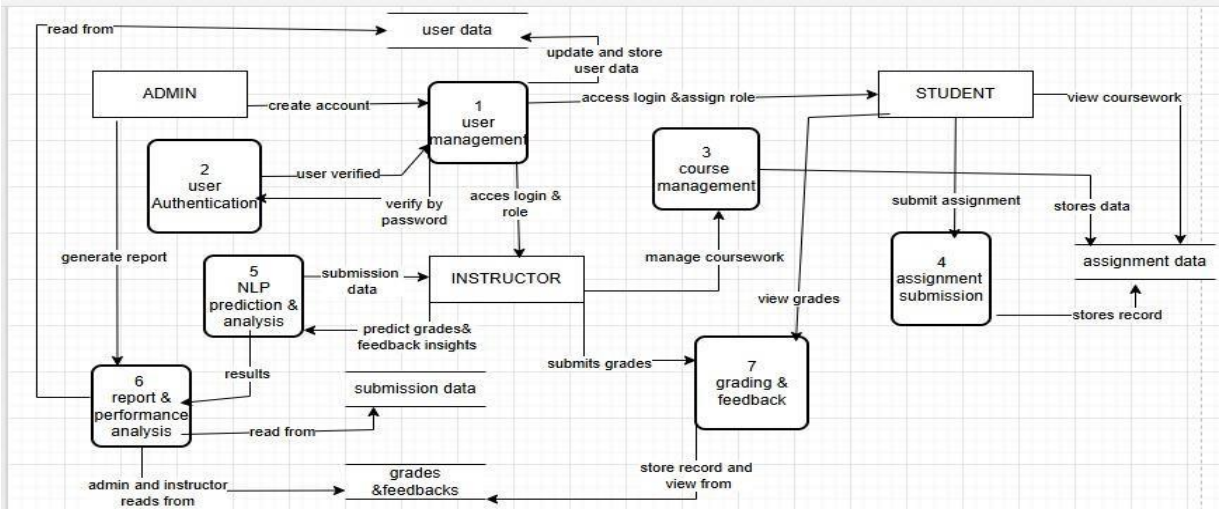


Figure 13: (DFD) Level 1 of the Automated Student Grading System

5.3 Data Design

5.3.1 Entity Relationship Diagram (ERD)

The below given ERD shows the logical structure of the database. It defines the main entities of the system, their attributes, and the relationships among them.

Entities & Attributes

1. Role → Id (PK), Name
2. Department → Id (PK), Nam
3. Semester → Id (PK), Name
4. Subject → Id (PK), Name
5. User → Id (PK)
6. UserLogin → Id (PK), UserId (FK), Username, Password, RoleId (FK)
7. Teacher → Id (PK), Name, Address, Email, Phone, SubjectId (FK), RoleId (FK), DepartmentId (FK), SemesterId (FK), Cnic
8. Student → Id (PK), Name, Address, Email, Phone, DepId (FK), RoleId (FK), SemesterId (FK), Cnic
9. SemesterSubject → Id (PK), SemesterId (FK), SubjectId (FK), DepId (FK), SubjectName
10. DepartmentSemester → Id (PK), DepId (FK), SemesterId (FK)
11. UploadDocument → Id (PK), Title, Description, FilePath, TeacherId (FK), DepId (FK), SemesterId (FK), SubjectId (FK), DueDate, UploadedDate, FileSize, FileName
12. StudentDocument → Id (PK), Title, Description, FilePath, StudentId (FK), TeacherDocumentId (FK), DepId (FK), SemesterId (FK), SubjectId (FK), UploadedDate, FileSize, FileName
13. TeacherStudentChat → Id (PK), Message, TeacherId (FK), StudentId (FK), Status, Sender
14. SingleResult → Id (PK), StudentId (FK), TeacherId (FK), DepId (FK), SubjectId (FK), SemesterId (FK), AnswerId, QuestionId, Feedback, PredictedMarks
15. PredictResult → Id (PK), StudentId (FK), DepId (FK), SemesterId (FK), SubjectId (FK), QuizMarks, AttendanceMarks, MidMarks, FinalMarks, AssignmentMarks, TotalMarks, EvaluationNotes, EvaluationDate, GP, Grade

Key Relationships, Business Rules, Type, Degree & Cardinality (ERD)

1. USER ---- USERLOGIN

Each USER must have exactly one USERLOGIN account.

Each USERLOGIN is tied to exactly one USER.

- Type of Relationship: One-to-One
- Degree of Relationship: Binary
- Cardinality: 1:1

2. ROLE ---- USERLOGIN

A ROLE can be assigned to many USERLOGIN accounts.

Each USERLOGIN must belong to exactly one ROLE.

- Type of Relationship: One-to-Many
- Degree of Relationship: Binary
- Cardinality: 1:N

3. ROLE ---- TEACHER

A ROLE (e.g., Teacher Role) can be assigned to multiple TEACHER records.

Each TEACHER must belong to exactly one ROLE.

- Type of Relationship: One-to-Many
- Degree of Relationship: Binary
- Cardinality: 1:N

4. ROLE ---- STUDENT

A ROLE (e.g., Student Role) can be assigned to multiple STUDENT records.

Each STUDENT must belong to exactly one ROLE.

- Type of Relationship: One-to-Many
- Degree of Relationship: Binary
- Cardinality: 1:N

5. DEPARTMENT ---- TEACHER

A DEPARTMENT can have many TEACHERS.

Each TEACHER belongs to exactly one DEPARTMENT.

- Type of Relationship: One-to-Many
- Degree of Relationship: Binary
- Cardinality: 1:N

6. DEPARTMENT ---- STUDENT

A DEPARTMENT can have many STUDENTS.

Each STUDENT must belong to exactly one DEPARTMENT.

- Type of Relationship: One-to-Many
- Degree of Relationship: Binary
- Cardinality: 1:N

7. SEMESTER ---- TEACHER

A SEMESTER can include many TEACHERS.

Each TEACHER must be linked to exactly one SEMESTER.

- Type of Relationship: One-to-Many
- Degree of Relationship: Binary
- Cardinality: 1:N

8. SEMESTER ---- STUDENT

A SEMESTER can include many STUDENTS.

Each STUDENT must be linked to exactly one SEMESTER.

- Type of Relationship: One-to-Many
- Degree of Relationship: Binary
- Cardinality: 1:N

9. SUBJECT ---- TEACHER

A SUBJECT can be taught by multiple TEACHERS.

Each TEACHER teaches exactly one SUBJECT (in this schema).

- Type of Relationship: One-to-Many
- Degree of Relationship: Binary
- Cardinality: 1:N

10. SEMESTER ---- SUBJECT (via SemesterSubject

A SEMESTER can have multiple SUBJECTS.

A SUBJECT can be offered in multiple SEMESTERS.

- Type of Relationship: Many-to-Many (resolved through SemesterSubject).
- Degree of Relationship: Ternary (via bridge table).
- Cardinality: M:N

11. DEPARTMENT ---- SEMESTER (via DepartmentSemester)

A DEPARTMENT can offer many SEMESTERS

A SEMESTER can belong to many DEPARTMENTS.

- Type of Relationship: Many-to-Many (resolved through DepartmentSemester).
- Degree of Relationship: Ternary (via bridge table).
- Cardinality: M:N

12. TEACHER ---- UPLOADDOCUMENT

A TEACHER can upload many documents.

Each UPLOADDOCUMENT must belong to exactly one TEACHER

- Type of Relationship: One-to-Many
- Degree of Relationship: Binary
- Cardinality: 1:N

13. STUDENT ---- STUDENTDOCUMENT

A STUDENT can submit many documents.

Each STUDENTDOCUMENT must belong to exactly one STUDENT.

- Type of Relationship: One-to-Many
- Degree of Relationship: Binary
- Cardinality: 1:N

14. TEACHER ---- STUDENT (via TeacherStudentChat)

A TEACHER can chat with many STUDENTS.

A STUDENT can chat with many TEACHERS.

- Type of Relationship: Many-to-Many (resolved via TeacherStudentChat).
- Degree of Relationship: Binary (through link table).
- Cardinality: M:N

15. STUDENT ---- PREDICTRESULT

A STUDENT can have multiple predictive results.

Each PREDICTRESULT belongs to exactly one STUDENT.

- Type of Relationship: One-to-Many
- Degree of Relationship: Binary
- Cardinality: 1:N

Table 2: Strong and weak entities of ERD

Strong Entities	Weak Entities
Role	UserLogin
Semester	SemesterSubject
Department	DepartmentSemester
Subject	UploadDocument
User	StudentDocument
Teacher	TeacherStudentChat
Student	PredictResult

ERD:

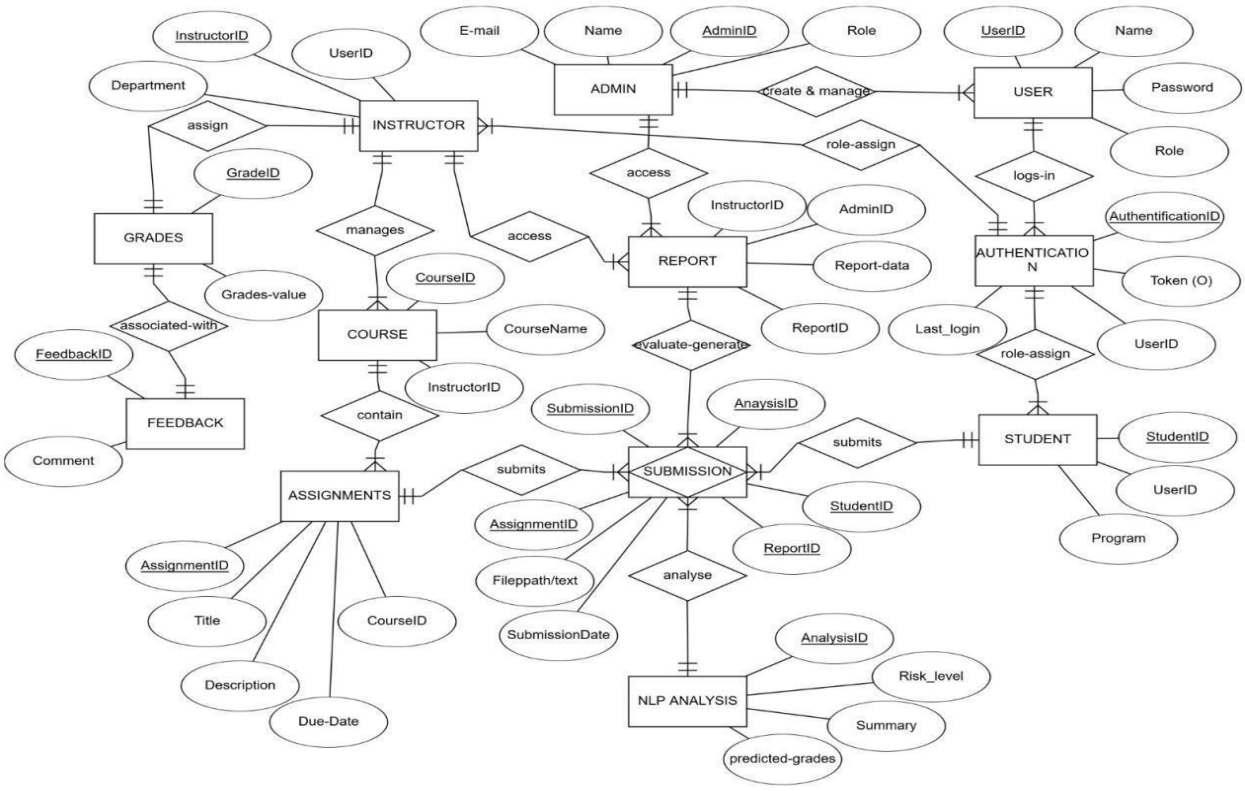


Figure 14: ERD

5.3.2 Database Schema

The below schema diagram represents the physical design of the database. It maps the entities into

relational tables with primary keys, foreign keys, and constraints.

Tables & Descriptions

1. Role

Fields: Id (PK), Name

Description: Defines roles such as Admin, Teacher, Student.

2. Department

Fields: Id (PK), Name

Description: Represents academic departments.

3. Semester

Fields: Id (PK), Name

Description: Defines semesters offered in the institution.

4. Subject

Fields: Id (PK), Name

Description: Represents subjects taught under departments and semesters.

5. User / UserLogin

Fields: User (Id), UserLogin (Id, UserId FK, Username, Password, RoleId FK

Description: Stores system users and their login credentials.

6. Teacher

Fields: Id (PK), Name, Address, Email, Phone, SubjectId (FK), RoleId (FK), DepartmentId (FK), SemesterId (FK), Cnic

Description: Stores teacher information with department, semester, and subject assignment.

7. Student

Fields: Id (PK), Name, Address, Email, Phone, DepId (FK), RoleId (FK), SemesterId (FK), Cnic

Description: Stores student information.

8. SemesterSubject / DepartmentSemester

Fields: Links between Semester–Subject–Department

Description: Bridge tables to resolve many-to-many relationships.

9. UploadDocument

Fields: Id (PK), Title, Description, FilePath, TeacherId (FK), DepId (FK), SemesterId (FK), SubjectId (FK), DueDate, UploadedDate, FileSize, FileName

Description: Assignments or documents uploaded by teachers.

10. StudentDocument

Fields: Id (PK), Title, Description, FilePath, StudentId (FK), TeacherDocumentId (FK), DepId (FK), SemesterId (FK), SubjectId (FK), UploadedDate, FileSize, FileName

Description: Documents submitted by students in response to uploads.

11. TeacherStudentChat

Fields: Id (PK), Message, TeacherId (FK), StudentId (FK), Status, Sender

Description: Facilitates communication between teachers and students.

12. SingleResult

Fields: Id (PK), StudentId (FK), TeacherId (FK), DepId (FK), SubjectId (FK), SemesterId (FK), AnswerId, QuestionId, Feedback, PredictedMarks

Description: Stores assessment results per student per subject.

13. PredictResult

Fields: Id (PK), StudentId (FK), DepId (FK), SemesterId (FK), SubjectId (FK), QuizMarks, AttendanceMarks, MidMarks, FinalMarks, AssignmentMarks, TotalMarks, EvaluationNotes, EvaluationDate, GP, Grade

Description: Stores predicted results and performance analytics for students.

Diagram:

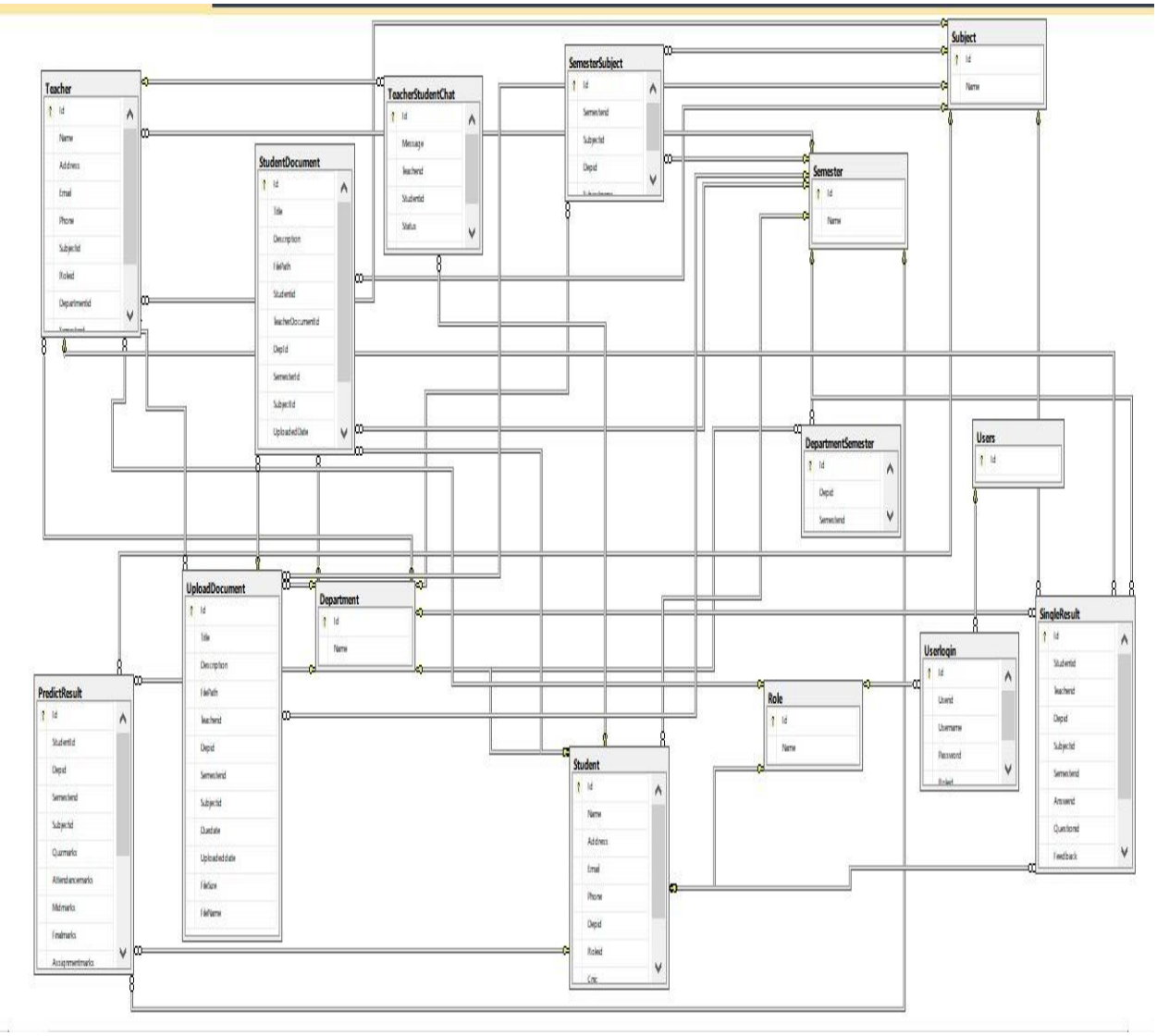


Figure 15: Schema Diagram of Automated Student Grading System

5.4 Interface Design

5.4.1 Admin Interfaces

1. Login Interface

Description:

This interface allows users (Admin, Teacher, and Student) to securely access the system by entering their credentials. It provides fields for username and password with a simple, user-friendly design. The login page ensures authorized access to the system and prevents unauthorized usage.

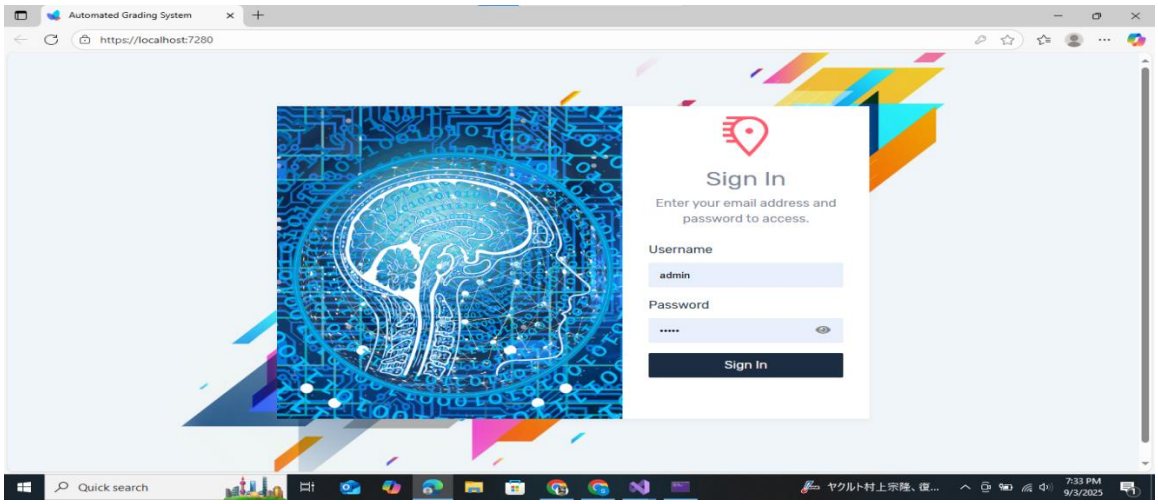


Figure 16: User Authentication Screen

2. Admin Dashboard Interface

Description:

The Admin Dashboard serves as the main control panel of the system, offering administrators both an overview of critical information and access to all core management modules. The top section of the dashboard provides a quick summary of the total number of students, teachers, and departments, enabling administrators to monitor the institution's overall status at a glance. On the left-hand side, the navigation panel is organized into different sections, with the Dashboard providing only a high-level view without any data-entry forms. The most significant section is Registration, which contains multiple submodules for managing institutional data. These submodules and their respective forms include:

- Department: Add Department, Edit Department, View Details
- Subject: Add Subject, Edit Subject, View/Delete
- Semester-Wise Subject: Assign Subjects to Semester/Department, Edit/Remove mapping
- Teacher: Add Teacher, Edit Teacher, View/Delete
- Student: Add Student, Edit Student, View/Delete

In addition to Registration, the dashboard also provides access to the Results module, where administrators can generate, upload, view, and publish examination results. Overall, the Admin Dashboard functions as both a monitoring tool and a centralized gateway for handling academic, administrative, and assessment-related tasks efficiently.

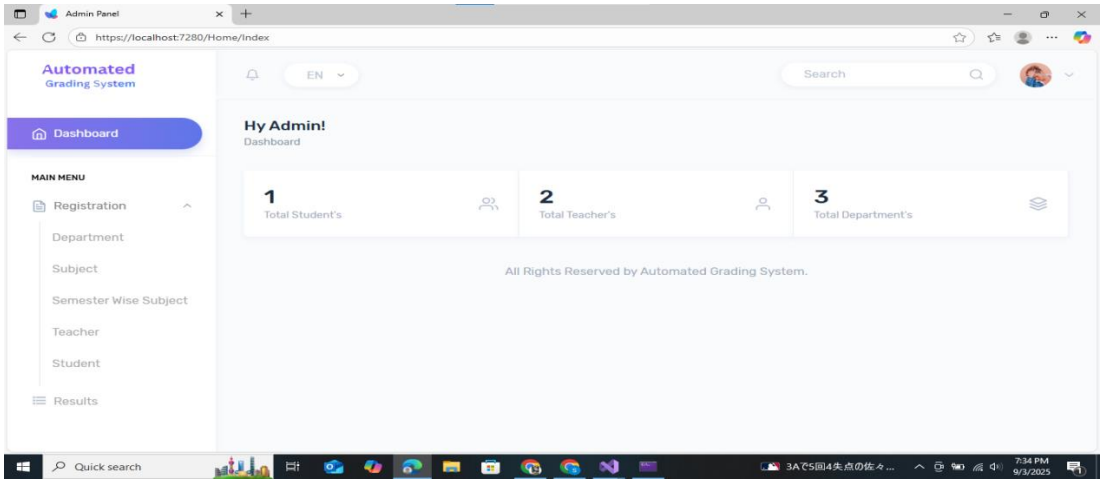


Figure 17: Admin Control Panel

3. Department Management Interface

Description:

The Department Management Interface allows administrators to view, add, edit, and delete academic departments. It displays all departments in a table with options to search, filter, and navigate records. A dedicated "Add New" button enables the creation of new departments, while action buttons provide quick access to edit or remove existing records.

Key functions include:

- Add Department: Create a new department with required details.
- Edit Department: Update information of an existing department.
- View Details: (Optional) Check complete department information.
- Delete Department: Remove a department with confirmation.
- Search & Pagination: Easily locate and manage records.

This interface ensures accurate and efficient management of departmental data within the system

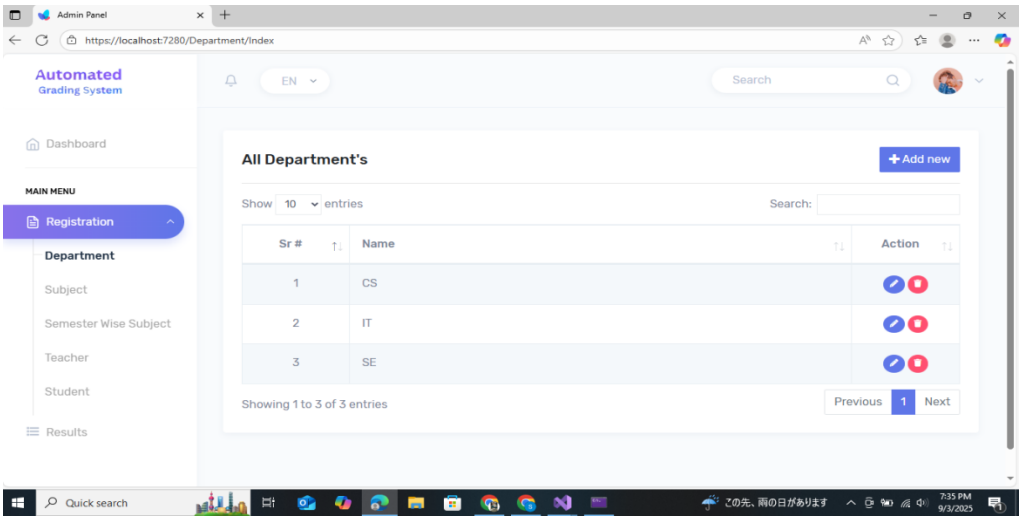


Figure 18: Department Management Screen

4. Subject Management Interface

Description:

The Subject Management Interface provides administrators with tools to manage all academic subjects. Subjects are displayed in a tabular format with details such as subject name and code. The interface includes a search bar and pagination for quick navigation. A dedicated “Add New” button allows the creation of new subjects, while action buttons enable editing or deleting existing records.

Key functions include:

- Add Subject: Create a new subject by entering subject name and details.
- Edit Subject: Update information of an existing subject.
- View/Delete Subject: Review details or remove a subject if required.
- Search & Pagination: Locate and manage subjects efficiently.

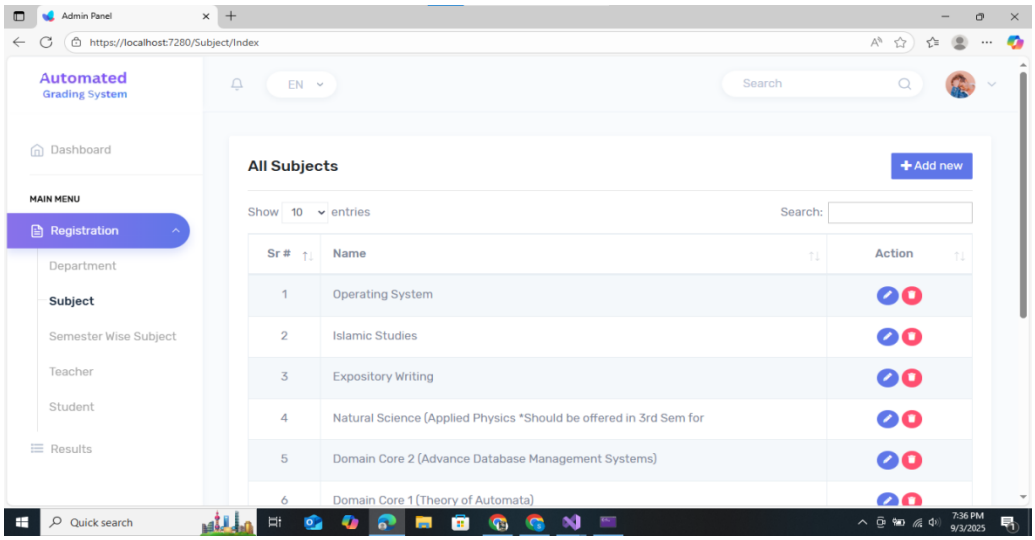


Figure 19: Subject Management Screen

5. Semester-Wise Subject Interface

The Semester-Wise Subject Interface is specifically designed to manage the allocation of subjects across different departments and semesters. This feature ensures that each semester has the correct subjects assigned, which is essential for structured course planning and curriculum management. The interface displays a table listing the department, semester, and assigned subjects, making it easy for administrators to review existing allocations. A search bar and pagination controls are included to handle large datasets efficiently. The presence of an "Add New" button allows administrators to create fresh mappings between subjects and semesters, while action buttons enable quick editing or removal of existing records.

Key functions include:

- Assign Subject to Semester/Department: Create new mappings by selecting a department, semester, and corresponding subject(s).
- Edit/Remove Mapping: Update incorrect allocations or delete outdated mappings to maintain accurate curriculum planning.
- Search & Pagination: Easily locate specific department-semester assignments and manage records effectively.

In short, this interface plays a vital role in ensuring that subjects are correctly distributed within each department's academic structure, maintaining consistency and reducing the chance of scheduling errors.

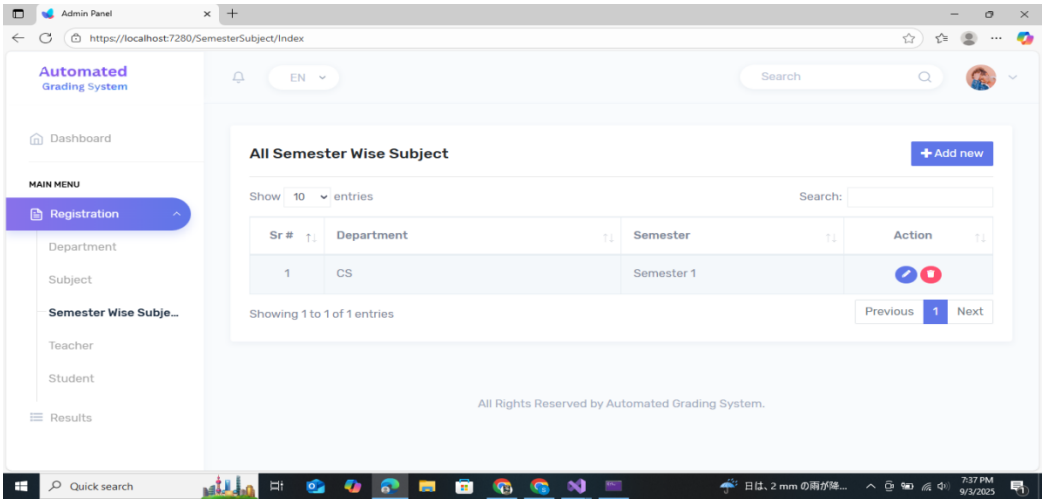


Figure 20: Semester-Wise Subject Screen

6. Teacher Management Interface

Description:

The Teacher Management Interface enables administrators to efficiently handle all teacher-related information within the system. It presents teacher records in a tabular format, displaying key details such as name, address, email, and phone number. This organized view allows administrators to quickly review existing data and perform necessary actions. A dedicated “Add New” button is provided for creating new teacher profiles, while each record in the table includes action buttons to edit or delete entries. The search bar and pagination controls ensure that specific records can be easily located, even when the dataset grows larger.

Key functions include:

- Add Teacher: Register a new teacher by entering personal and professional details such as name, address, email, and phone number.
- Edit Teacher: Update or correct information for an existing teacher.
- View/Delete Teacher: Access complete details of a teacher or permanently remove their record from the system.
- Search & Pagination: Locate individual teacher records quickly and manage large datasets effectively.

In summary, this interface streamlines the management of teacher information and ensures that academic staff records remain accurate, updated, and easy to access.

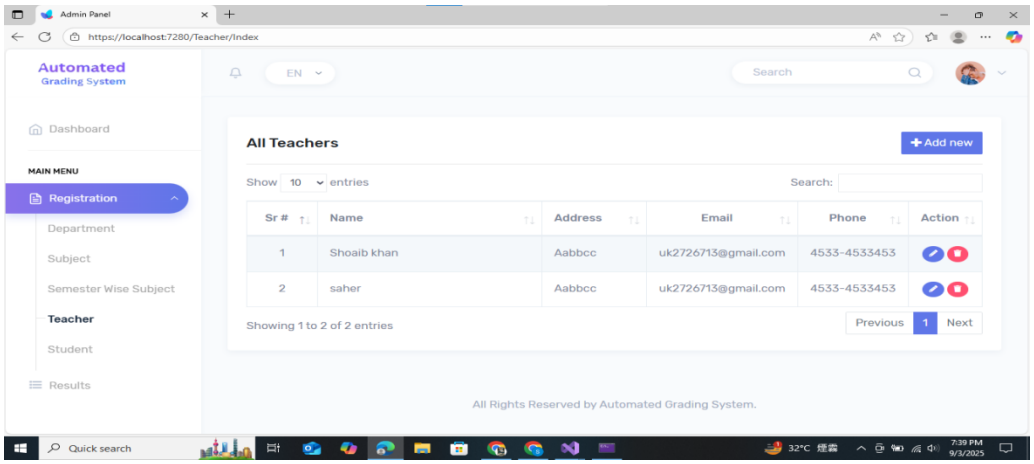


Figure 21: Teacher Management Screen

7. Student Management Interface

Description:

The Teacher Management Interface enables administrators to efficiently handle all teacher-related information within the system. It presents teacher records in a tabular format, displaying key details such as name, address, email, and phone number. This organized view allows administrators to quickly review existing data and perform necessary actions. A dedicated “Add New” button is provided for creating new teacher profiles, while each record in the table includes action buttons to edit or delete entries. The search bar and pagination controls ensure that specific records can be easily located, even when the dataset grows larger.

Key functions include:

- Add Teacher: Register a new teacher by entering personal and professional details such as name, address, email, and phone number.
- Edit Teacher: Update or correct information for an existing teacher.
- View/Delete Teacher: Access complete details of a teacher or permanently remove their record from the system.
- Search & Pagination: Locate individual teacher records quickly and manage large datasets effectively.

In summary, this interface streamlines the management of teacher information and ensures that academic staff records remain accurate, updated, and easy to access.

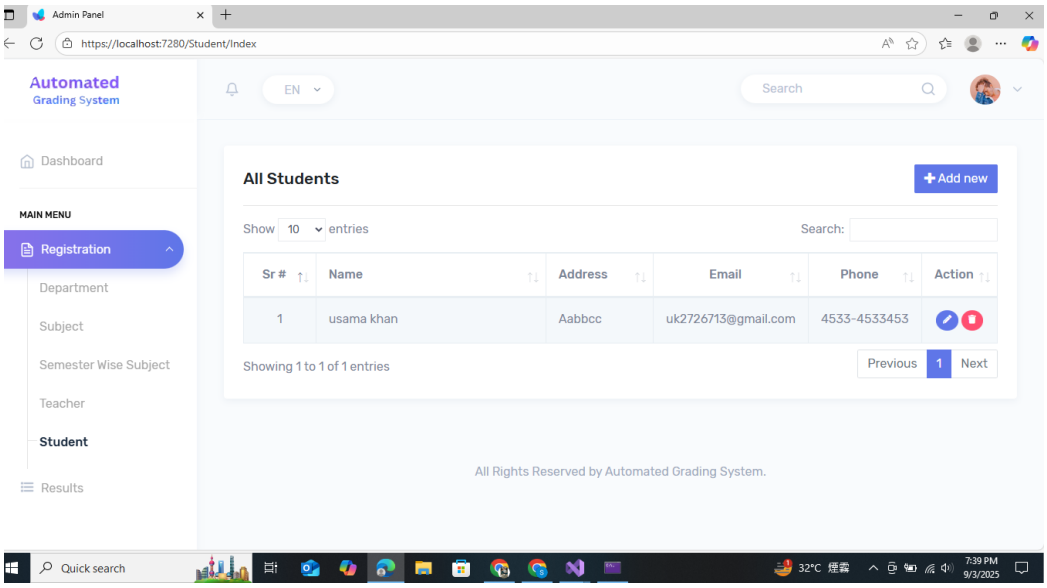


Figure 22: Student Management Screen

8. Result Management Interface

Description:

The Result Management Interface is designed to display and manage students' examination results. It presents results in a tabular format, organized semester-wise and subject-wise, making it easy to track academic performance over time. Administrators can search for any registered student and view the results that have been generated and submitted by teachers.

This interface provides quick access to student performance records and ensures that results remain accurate and updated. Each record includes action buttons that allow the administrator to view, update, or delete results whenever required.

Key functions include:

- View Results: Administrators can view results entered by teachers for registered students.
- Search Student: Locate a specific student's academic record instantly using the search functionality.
- Update/Delete Results: Modify incorrect entries or remove outdated results when necessary.
- Semester & Subject Filtering: Results are displayed systematically by semester and subject, ensuring clarity and easy navigation.
- Performance Tracking: Helps in maintaining a complete academic history of students for reporting and analysis.

In summary, this interface centralizes the management of examination results, allowing administrators to access, verify, and manage student performance data with efficiency.

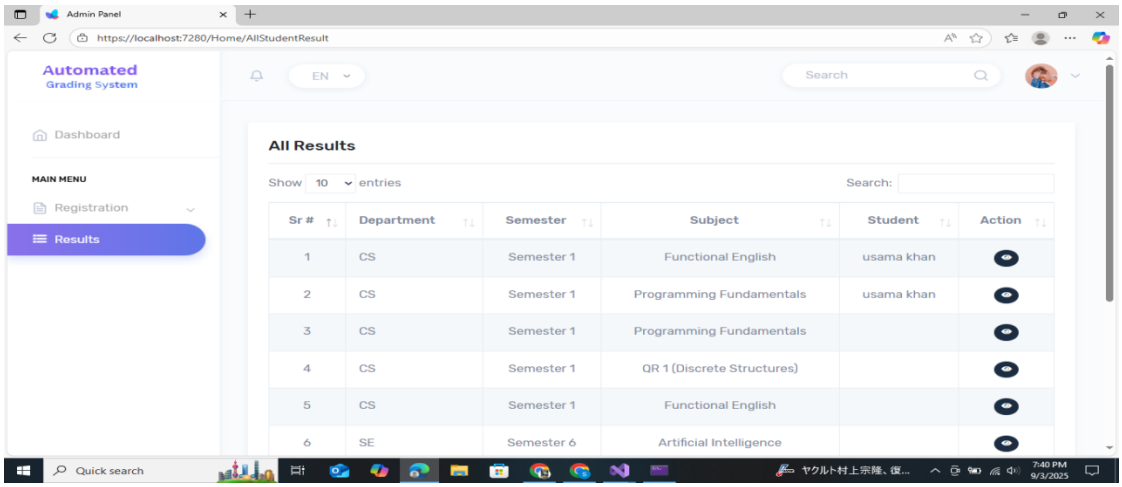


Figure 23: Result Management Screen

9. Detailed Result Entry & View Interface

Description:

The Result Entry Interface allows administrators or authorized teachers to add new examination results for registered students. It provides a structured form with dedicated fields such as Student Name, Semester, Subject, Total Marks, and Obtained Marks. This ensures accuracy and consistency while entering academic data.

The interface makes the process of recording results simple and efficient. By selecting a student from the registered database, teachers can quickly assign marks for specific subjects and semesters. Once saved, these results become part of the student's academic record and are immediately available for review in the Result Management Interface.

Key functions include:

- Add New Result: Enter results for a student by filling out details such as subject, semester, and marks.
- Select Registered Student: Choose students from the registered database to ensure accurate mapping of results
- Subject & Semester Mapping: Assign marks subject-wise and semester-wise for systematic record-keeping.
- Save & Update Records – Store new results or update previously saved ones as needed.

- Data Accuracy: Prevents duplication or mismatch of results by linking directly with the student database.

In summary, this interface streamlines the process of entering academic results, ensuring that student performance records remain accurate, organized, and easily accessible for both teachers and administrators.

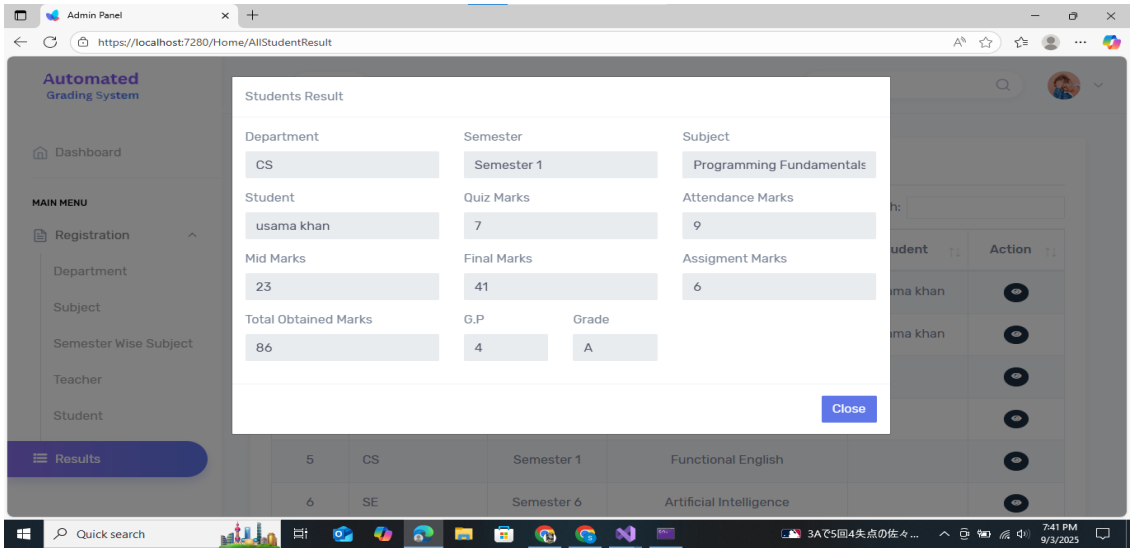


Figure 24: Detailed Result & View Screen

5.4.2 Teacher Interfaces

1. Teacher Login Interface

Description:

The Teacher Login Interface serves as the entry point for faculty members to access the system. The interface presents a clean and user-friendly design, consisting of a login form placed on the right-hand side and a graphical illustration on the left. The form contains two essential input fields: Username and Password, along with a “Sign In” button that initiates authentication. Teachers must provide valid credentials to proceed, ensuring that only authorized faculty members can gain access.

This interface also incorporates security by preventing unauthorized access to sensitive data, such as student records and academic results. The design emphasizes simplicity and usability, allowing teachers to log in quickly without unnecessary complexity. Once authenticated, the teacher is redirected to the Teacher Dashboard, where detailed teaching and performance-related modules are available.

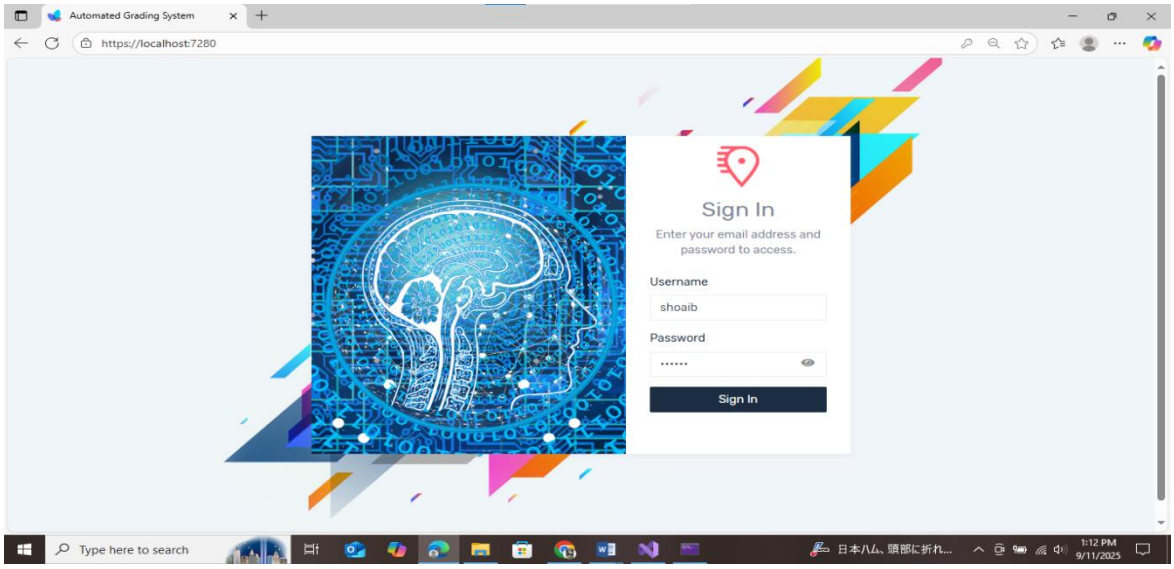


Figure 25: Teacher Authentication Screen

2. Teacher Dashboard Interface

Description:

The Teacher Dashboard serves as the main workspace for faculty members, providing both performance insights and easy navigation to teaching modules. The interface displays a bar chart and a pie chart, which visualize student performance and overall class statistics. These analytics, supported by NLP-based data processing, help teachers quickly identify learning trends and weak areas.

The sidebar menu gives structured access to key modules, including Profile Management, Assignments (for uploading and assigning tasks), and Results (for entering and reviewing student outcomes). With integrated search and navigation, teachers can manage their classes efficiently.

Overall, the dashboard combines analytics with direct access to core functions, enabling teachers to monitor performance, assign coursework, and maintain student records from a single platform.

Key functionalities include:

- View overall student performance through charts.
- Track subject-wise statistics.
- NLP based analysis
- Navigate quickly to Profile, Attendance, Assignments, and Results.

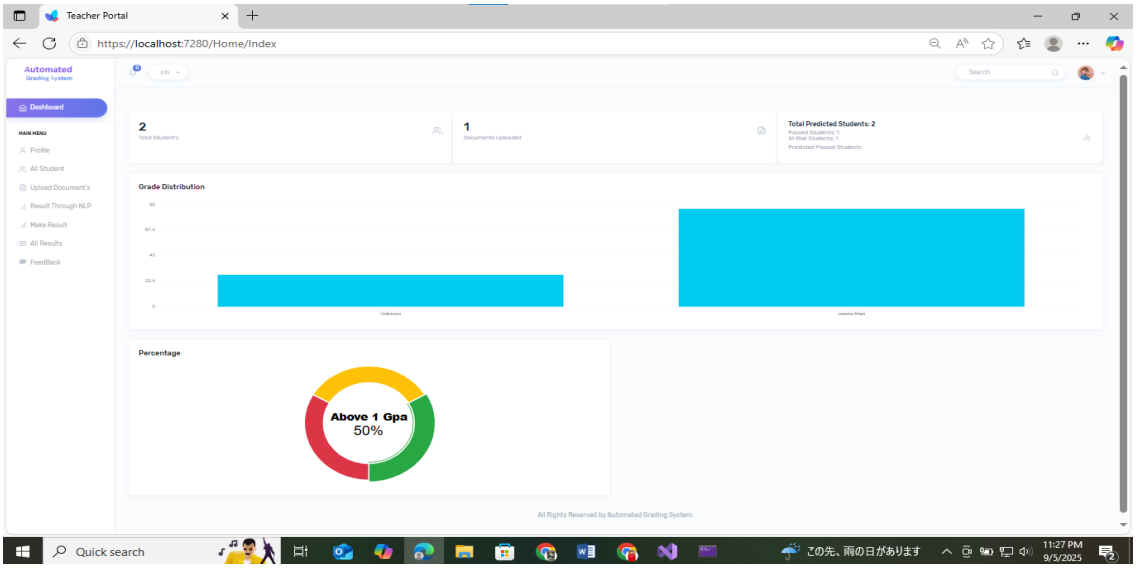


Figure 26: Teacher Dashboard Screen

3. Teacher Profile Management Interface

Description:

The Teacher Profile Management Interface enables faculty members to view and update their personal and academic details in a simple and structured way. The form displays fields such as Name, Address, Email, and Phone Number, along with academic details like Department and Designation. All fields are editable, ensuring that teachers can keep their information up to date. An “Update” button at the bottom allows changes to be saved instantly. This interface ensures that teacher records remain accurate, which helps both administrators and students access reliable contact and academic details.

Key functionalities include:

- View Profile: Display complete teacher details including personal and academic information.
- Update Personal Details: Modify name, address, email, or phone number.
- Save Changes: Apply updates instantly using the “Update” button.

In short, this interface acts as a self-service tool for teachers to manage their own records, reducing admin workload and keeping faculty information current.

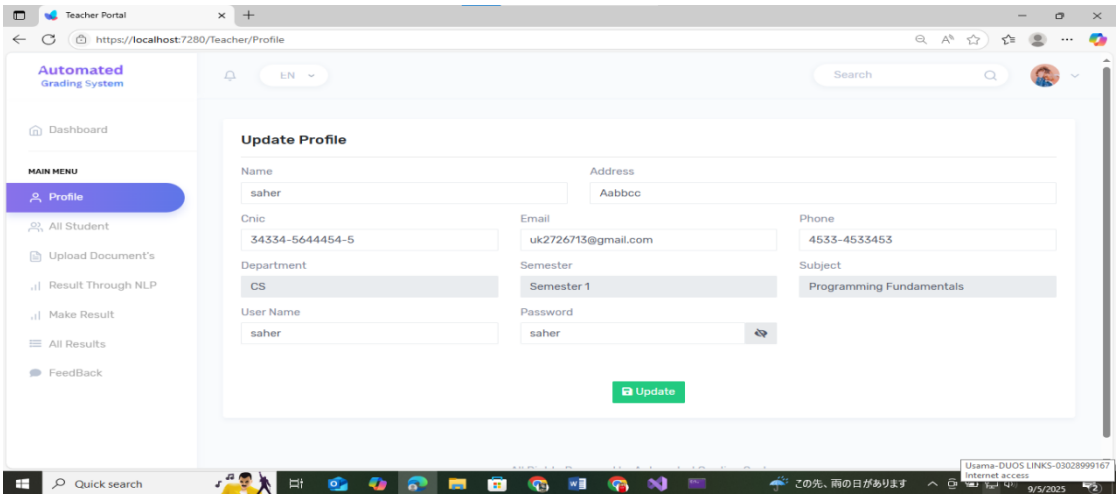


Figure 27: Teacher Profile Update Screen

4. Manage Subjects Interface

Description:

The Student Management Interface provides teachers with a complete list of students enrolled under their supervision. The table includes details such as student name, registration number, department, and semester. This centralized view helps teachers easily track student information for academic and administrative purposes. Search and pagination features make it convenient to locate individual student records.

Key functionalities include:

View Student Records: Displays all students assigned to the teacher's department/subjects.

Search & Pagination: Quickly locate specific students in large datasets.

Student Details Overview: Provides basic academic information (registration, department, semester).

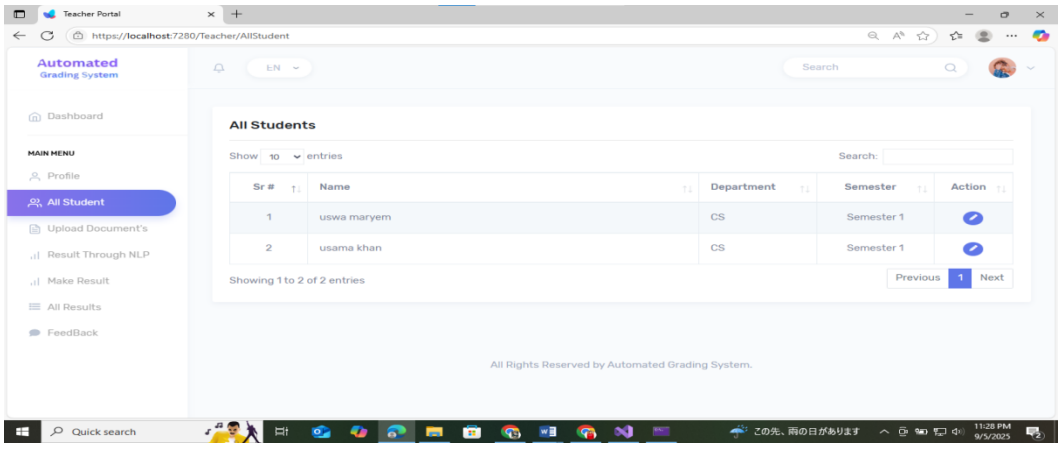


Figure 28: All Students Screen

5. Assignment Upload Interface

Description:

The Assignment Upload interface enables teachers to manage their coursework submissions digitally. Teachers can upload new assignments by specifying details such as title, department, semester, subject, file name, upload date, and due date. A table lists all uploaded assignments, with options for management.

Key functionalities include:

Upload New Assignment: Add assignments with subject and due-date details.

View Uploaded Assignments: See all previously uploaded assignments in a tabular view.

Edit/Delete: Modify or remove assignments if required.

Search & Pagination: Easily access a specific assignment.

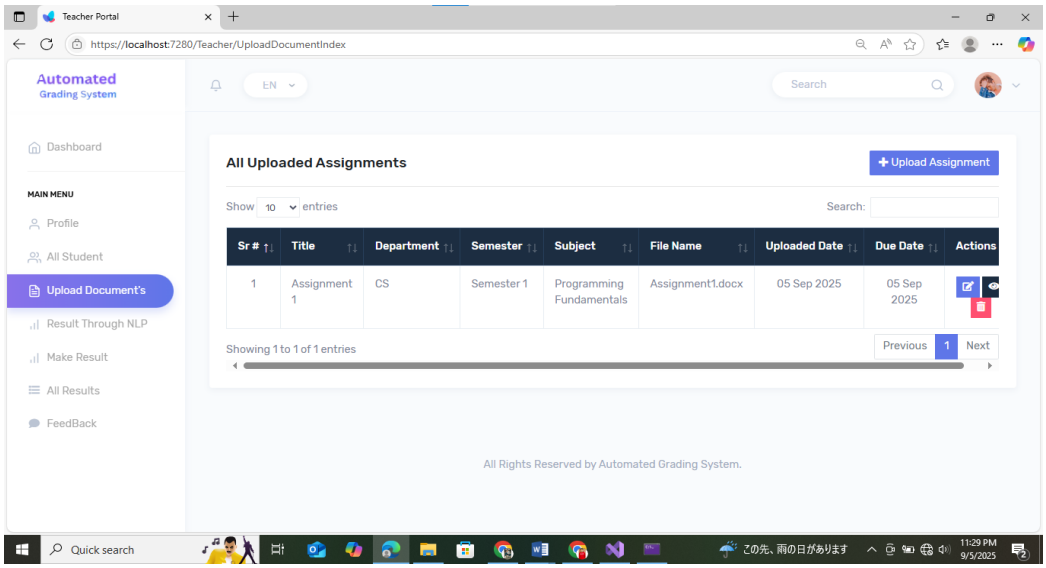


Figure 29: Teacher Assignment Upload Screen

6. Student Assignment Submission Interface

Description:

This interface provides teachers with a clear view of which students have submitted their assignments. A pop-up table lists student names along with their submission status (Yes/No). It allows faculty to track pending work and ensure timely submission compliance.

Key functionalities include:

View Student Submissions: Track who has submitted and who hasn't.

Submission Status (Yes/No): Quickly identify completed vs pending assignments.

Close/Navigation Option: Return back to the assignment list after review.

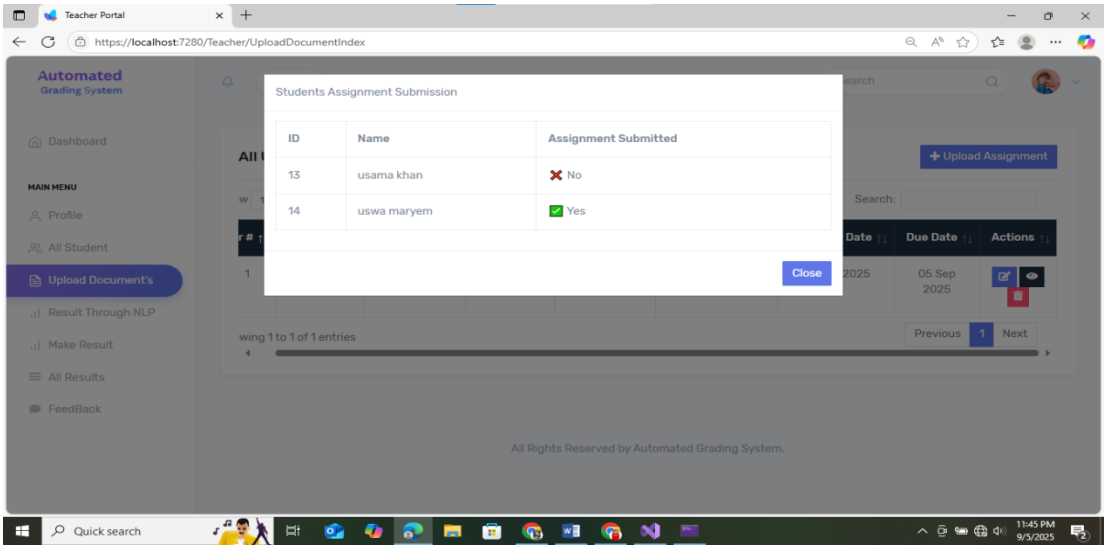


Figure 30: Student Submission Tracking Screen

7. Student Listing Interface

Description:

This interface displays a list of students enrolled in the teacher's courses. Each row contains student details along with an Action button, which allows the teacher to open the student's assignments for evaluation. This provides a quick way to select the student whose work needs to be graded.

Key functionality

- View all students.
- Use the Action button to open and check assignments.

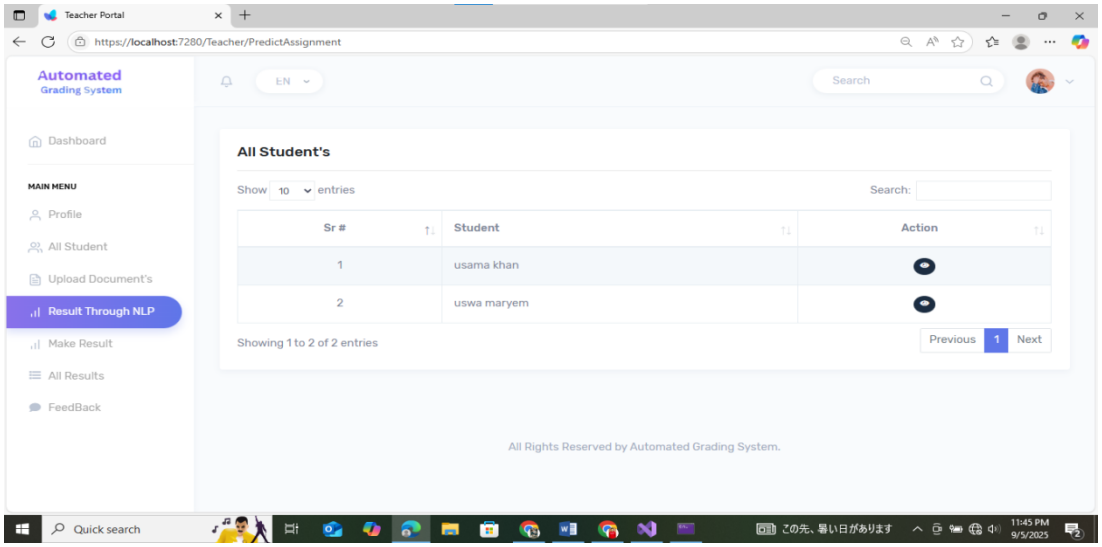


Figure 31: All Students Screen

8. NLP-Based Assignment Evaluation Interface

Description:

This interface allows teachers to evaluate assignments using NLP-based auto-grading. By clicking the red action button, teachers can access a student's submitted assignment. The system generates predicted marks automatically using NLP and displays the student's answer alongside the evaluated answer. Teachers also have the option to provide feedback or manually adjust marks, ensuring a balance between automation and teacher control.

Key functionality:

- Auto-grading through NLP prediction.
- View student answers vs evaluated answers.
- Add teacher's own feedback.
- Adjust marks manually if required.

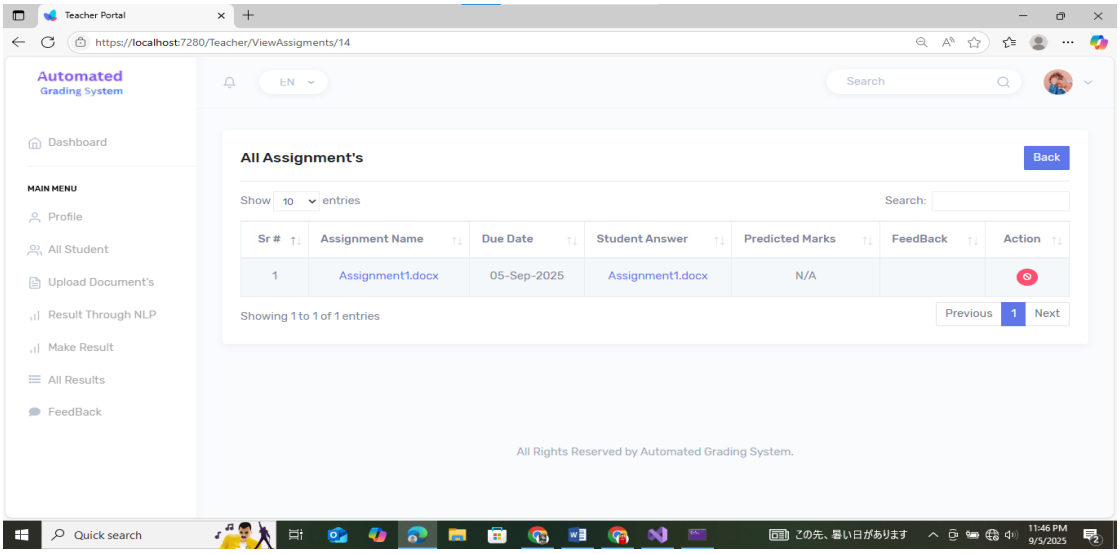


Figure 32: Assignment Auto-Grading Screen

9. Assignment Result Interface

Description:

After NLP evaluation, this interface displays the final results of student assignments. Teachers can review the predicted score, feedback, and evaluation status for each submission. This ensures transparency and helps teachers finalize grading efficiently.

Key functionality:

- View final predicted marks.
- Access feedback generated by NLP or teacher.

- Track evaluation status of all assignments.

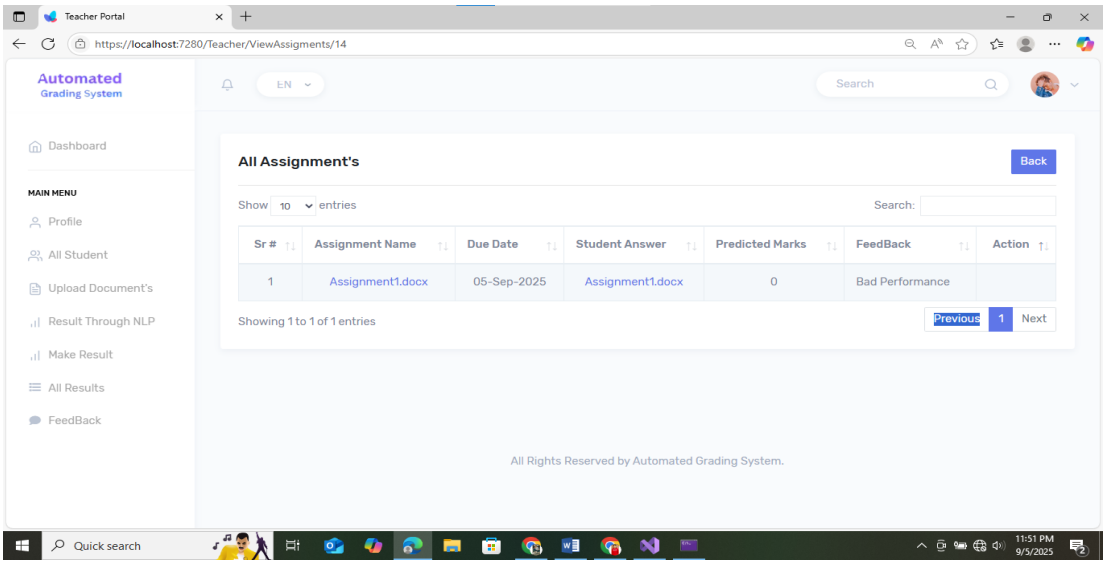


Figure 33: Assignment Result & Feedback Screen

10. Result Prediction & Entry Interface

Description:

This interface allows teachers to enter detailed marks for a selected student by choosing their department, semester, and subject. The marks entry form includes fields for:

- Quiz Marks (out of 7.5)
- Assignment Marks (out of 7.5)
- Attendance Marks (out of 10)
- Midterm Marks (out of 25)
- Final Term Marks (out of 50)

Based on these inputs, the system automatically calculates:

- Total Obtained Marks
- Final Grade
- GPA

Once the data is entered, the teacher/admin can save the result, and it will be stored in the system for future reference.

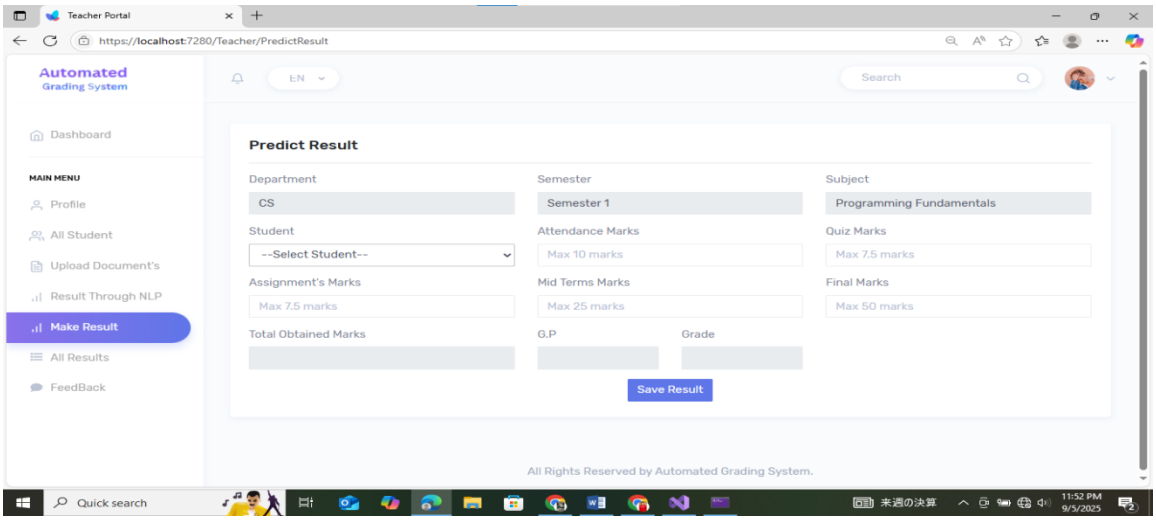


Figure 34: Result Entry & Grade Calculation Screen

11. Result Management Interface

Description:

This interface displays the list of all saved results in a tabular format. It organizes records by student, semester, and subject. Each entry includes an Action button (View Result) that allows the teachers to open detailed marks and grades of a particular student.

The search bar enables quick access to a specific student's record, while pagination ensures smooth navigation when the dataset is large.

Key functions include:

- Search for a registered student.
- Filter results by semester and subject.
- View complete result details by clicking the "View" button.

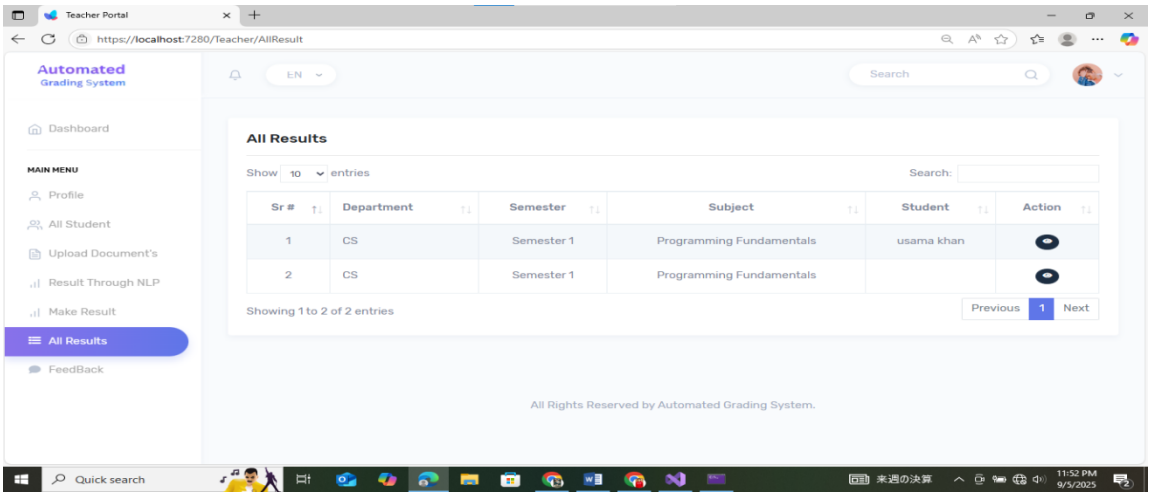


Figure 35: Result Record & Search Screen

12. Final Result View Interface

Description:

This interface presents the complete performance record of a selected student in detail. It shows department, semester, subject, and student name, along with all the marks breakdown:

- Quiz Marks
- Assignment Marks
- Attendance Marks
- Midterm Marks
- Final Marks
- Total Obtained Marks
- Final Grade
- GPA

The grading is based on the usual grading criteria followed by most universities. The system automatically converts the total obtained marks into a grade and GPA, ensuring consistency and fairness.

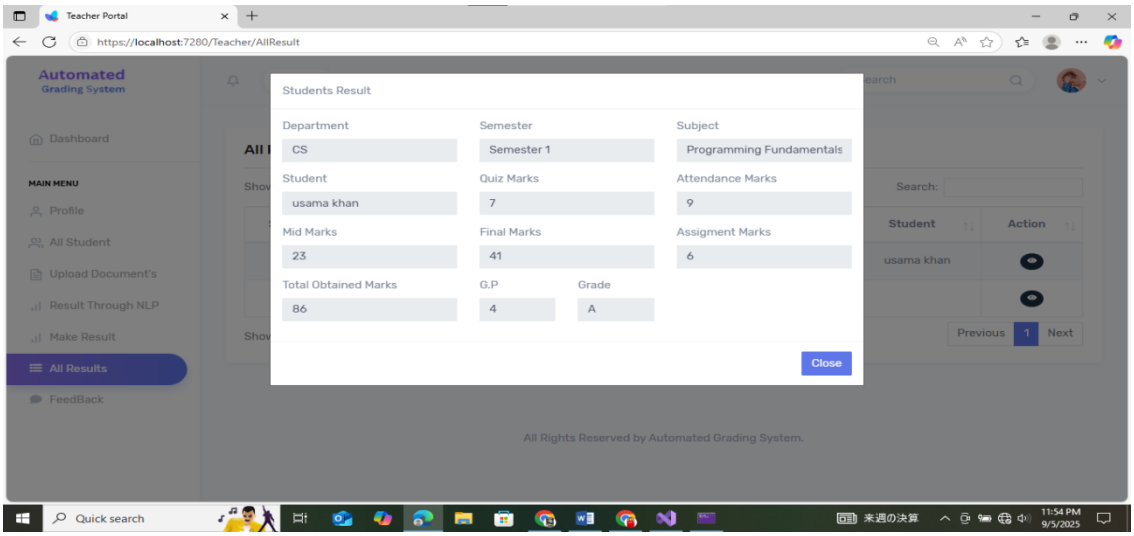


Figure 36: Detailed Student Subject Result Screen

13. At-Risk Students Interface

Description:

This interface allows teachers to identify and monitor students who are academically at risk. It displays a list of students with a GPA below 2.0, clearly highlighting those who need attention and support.

Each record in the table includes an Action button that enables the teacher to provide personalized feedback or alerts. This ensures that underperforming students are quickly recognized and guided.

Key functions include:

- Identify At-Risk Students: Automatically lists all students with a GPA below 2.0.
- Quick Feedback Access: Action button opens the feedback form for direct communication.
- Search & Pagination: Easily find specific students even in larger datasets.

In summary, this interface serves as a monitoring tool, helping teachers provide timely interventions to students who are struggling academically.

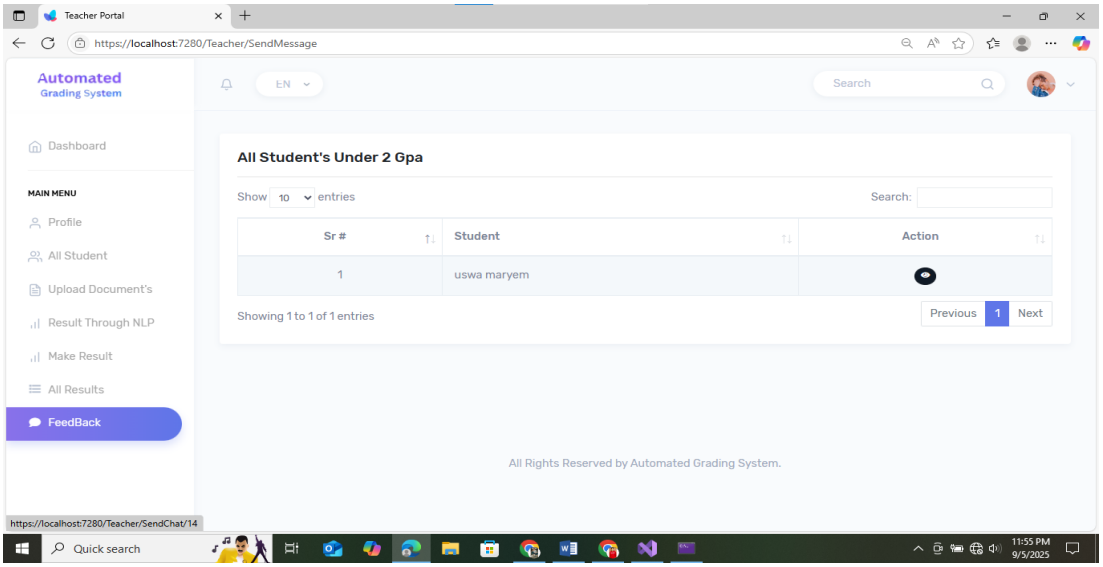


Figure 37: At-Risk Students Monitoring Screen

14. Teacher Feedback Interface

Description:

This interface enables teachers to send direct feedback messages to students flagged as underperforming. A simple text box allows the teacher to type customized feedback related to academic performance, attendance, or overall behavior.

In addition, the interface displays a record of previous feedback chats, ensuring both teacher and student can track communication history. This strengthens accountability and provides continuity in support.

Key functions include:

- Send Feedback/Alerts: Type and send performance-related feedback instantly.
- View Previous Feedback: Keep track of all past feedback messages for better follow-up.
- Direct Student Communication: Allows teachers to guide students effectively by giving clear and personalized instructions.

In summary, this interface streamlines teacher and student communication by making it easy to send alerts, track past feedback, and ensure that at-risk students receive the necessary academic support.

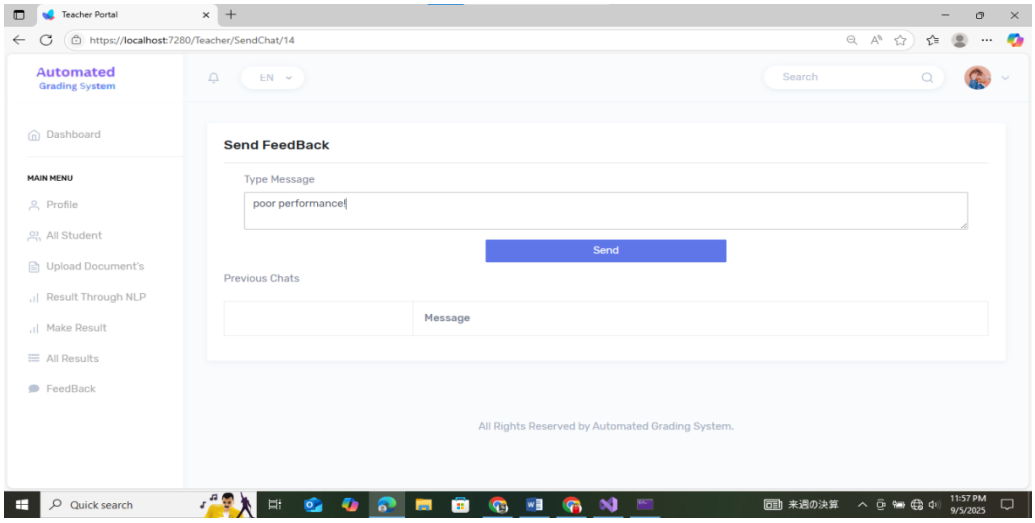


Figure 38: Feedback Communication Screen

5.4.3 Student Interfaces

1. Student Login Interface

Description:

Students log in with username and password to securely access their dashboard and academic data.

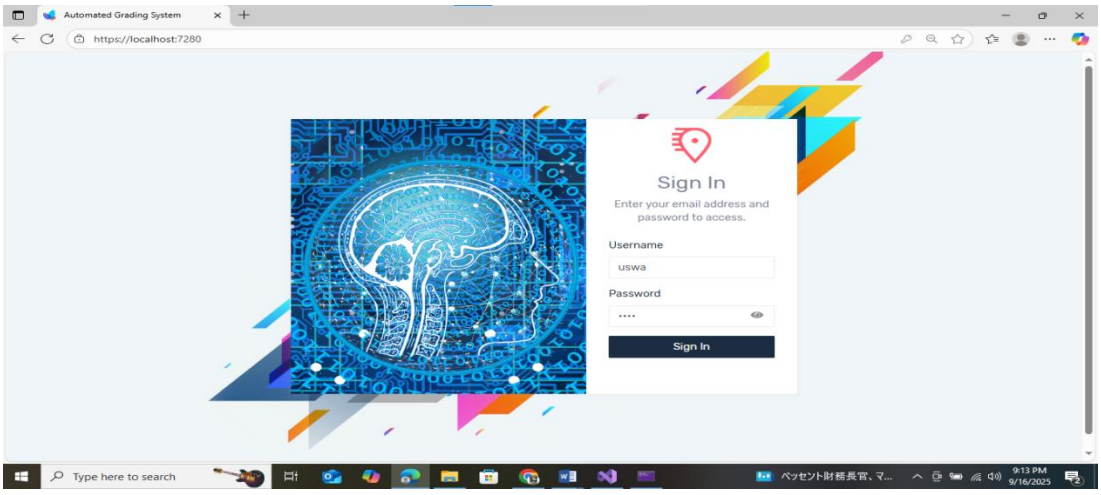


Figure 39: Student Authentication Screen

2. Student Profile Management Interface

Description:

Students can view and update personal details (name, email, phone, address) and academic info (department, semester). Updates are saved instantly.

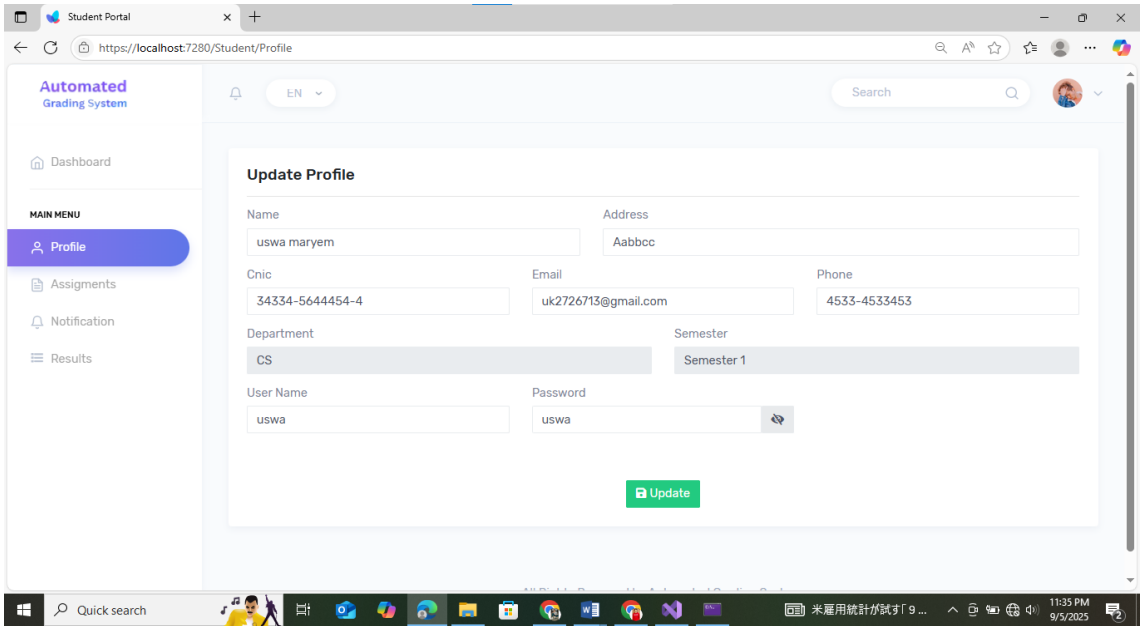


Figure 40: Profile Update Screen

3. Student Assignment Interface

Description:

The Student Assignment Interface provides a complete list of assignments uploaded by teachers. The table shows details such as assignment title, subject, semester, department, uploaded date, due date, and submission status. This helps students track deadlines and submit their work on time.

Key functionalities include:

- View All Assignments: Access assignments uploaded by teachers.
- Track Submission Status: See whether an assignment is pending or submitted.
- Due Date Monitoring: Check deadlines for timely submission.
- Search & Pagination: Quickly find specific assignments.

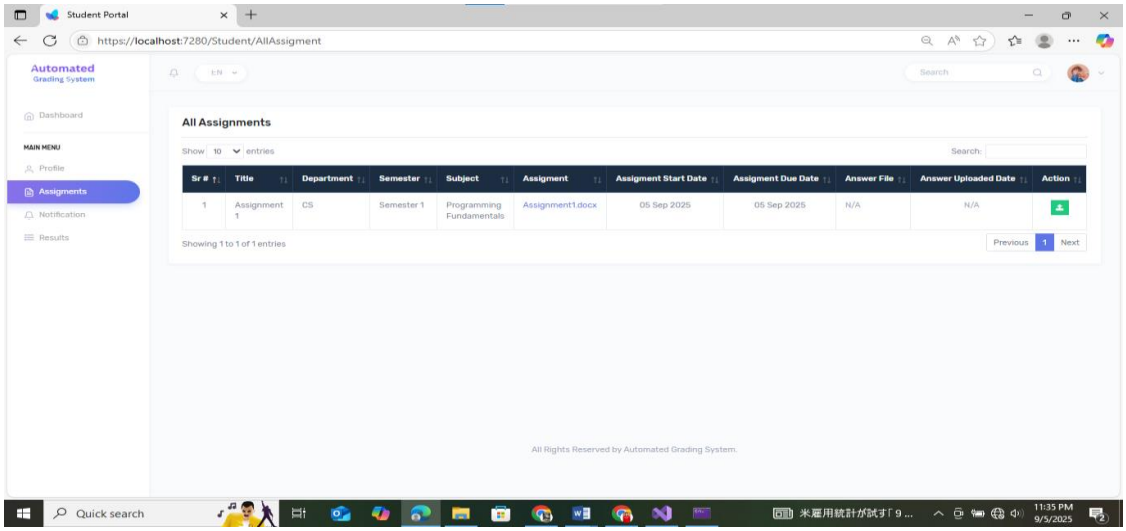


Figure 41: Assignment Overview Screen

4. Student Assignment Submission Interface

Description:

Students can upload their assignment answers. After a successful upload, a confirmation message (Success – Answer uploaded successfully) is displayed.

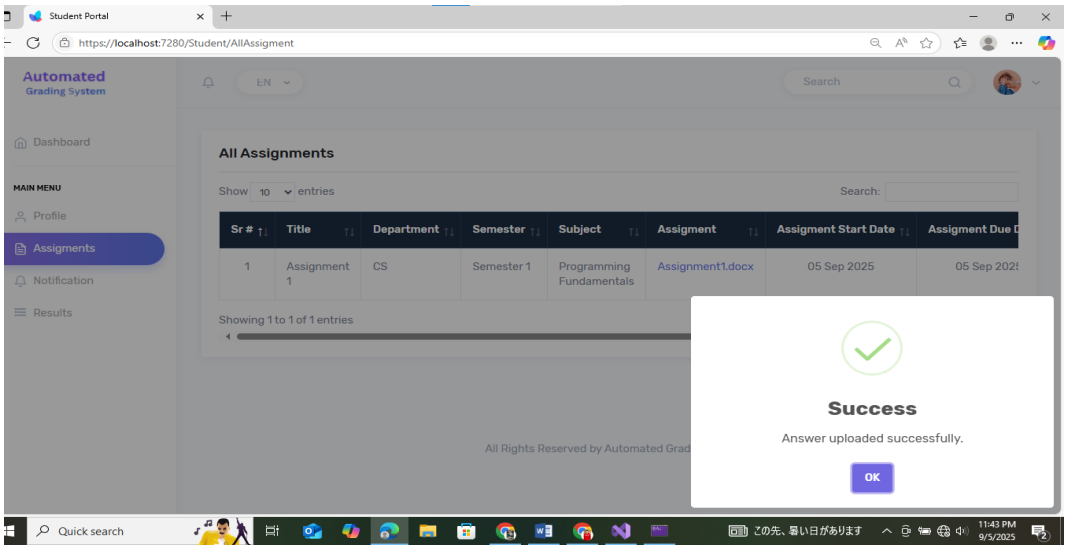


Figure 42: Assignment Upload Screen

5. Student Notification Interface

Description:

Students receive and view notifications or replies from teachers. The interface lists all teacher chats with options to open and review details.

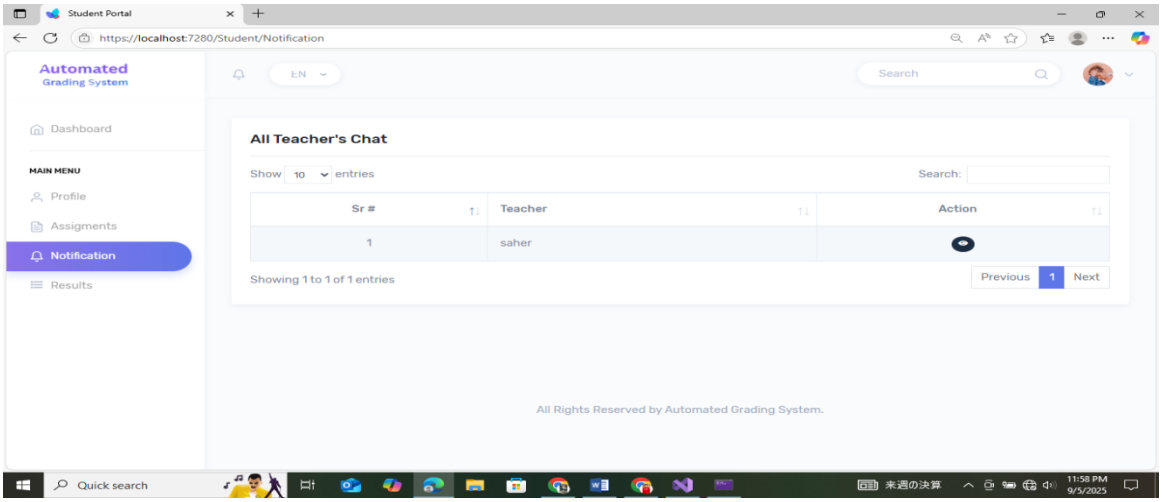


Figure 43: Notification/Teacher's Chat Screen

6. Student Chat Interface

Description**:**

This interface allows students to directly send messages to teachers. Previous chat history is shown for reference, supporting clear teacher-student communication.

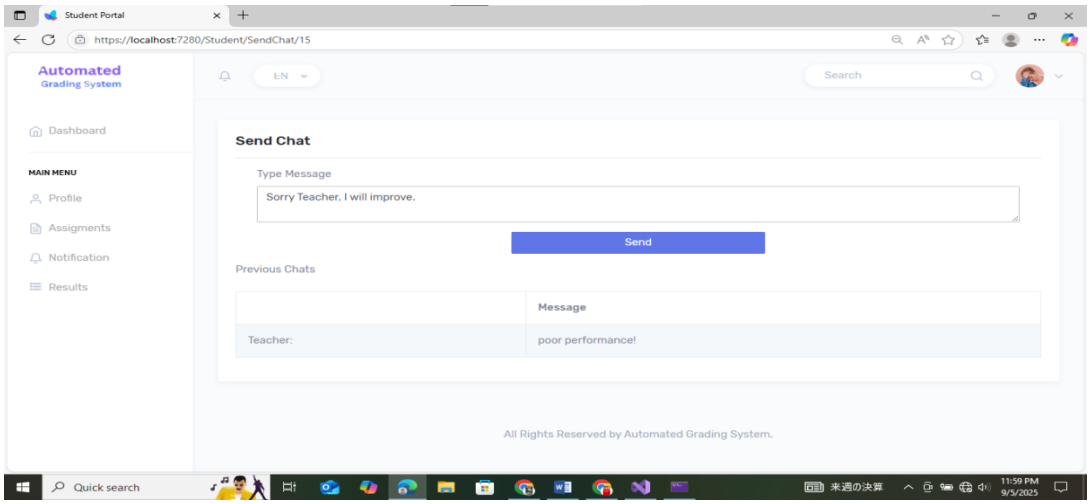


Figure 44: Chat with Teacher Screen

7. Student Result Listing Interface

Description:

This interface shows a list of results for the student, organized by semester and subject. Each record has an Action button that opens the detailed marks view.

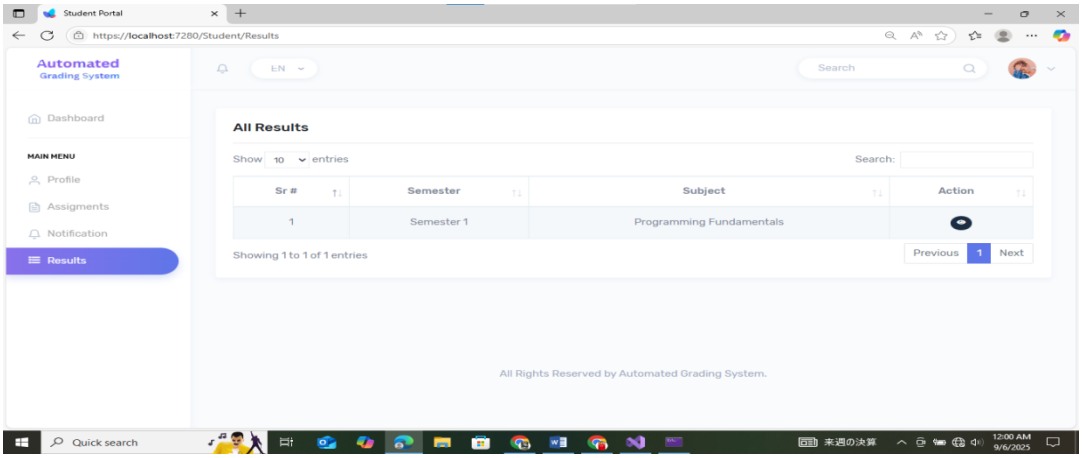


Figure 45: All Results Screen

8. Student Detailed Result Interface

Description:

When the student clicks View, a pop-up displays complete result details, including Midterm, Final, Attendance, and Assignment marks. The system automatically calculates total marks, GPA, and grade based on predefined university grading criteria.

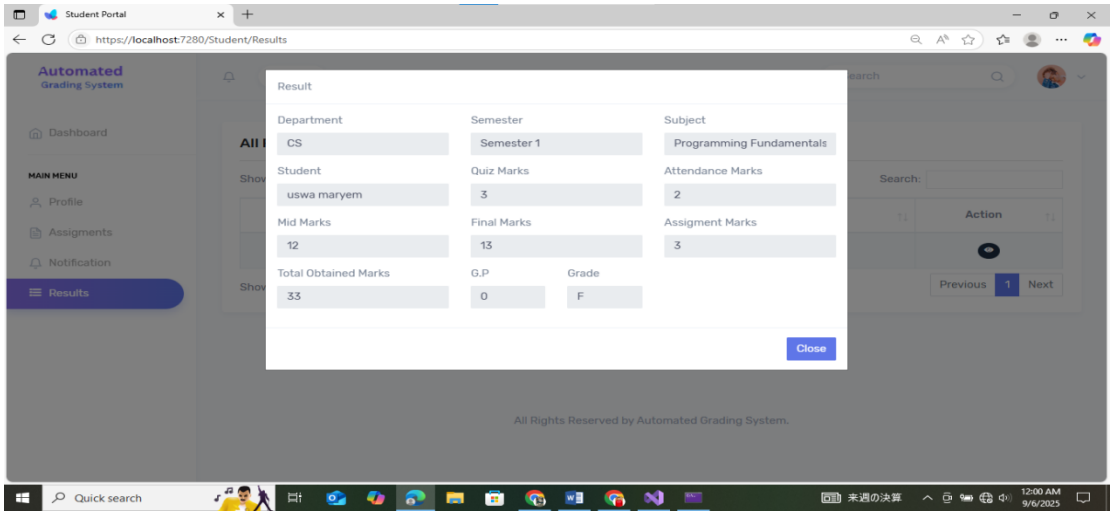


Figure 46: Result Details Screen

6. Results and Discussion

6.1 Results

This section presents the results of the developed system. The outcomes are demonstrated through screenshots, database outputs, and system interfaces. Each result corresponds to the project objectives and shows how the implemented system fulfills the functional requirements. The results include user authentication, role management, Results prediction through NLP, document uploads, communication modules, Notification and feedback.

1. Login and Authentication

The system provides a secure login feature where three types of users Admin, Teacher, and Student can sign in using their respective credentials.

- If the username and password are correct, the user is successfully authenticated and redirected to their respective dashboard according to their role.
- If the credentials are invalid, the system generates an “Invalid Username or Password” error message to prevent unauthorized access.

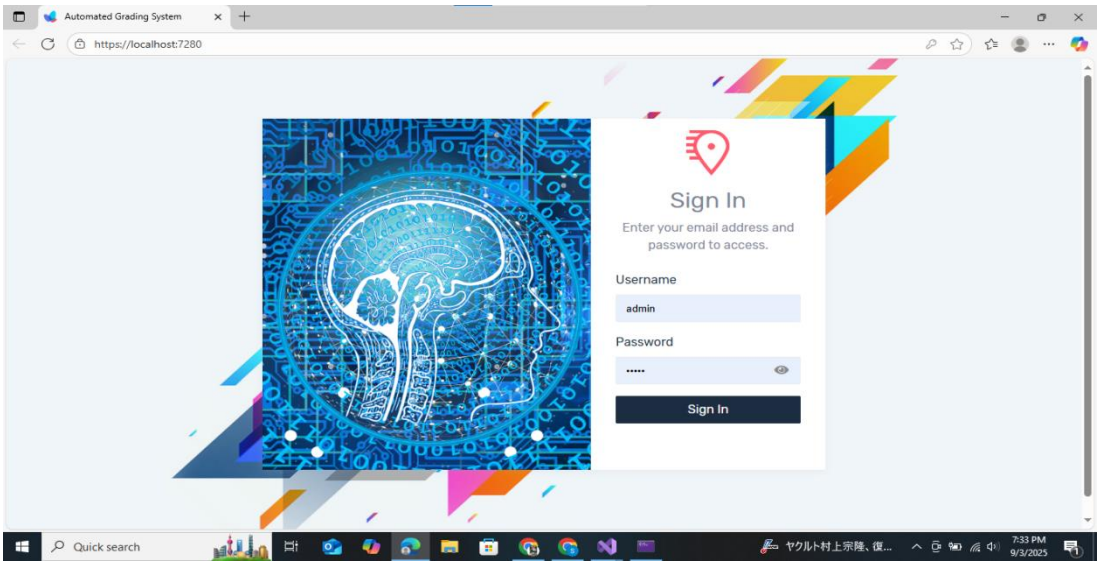


Figure 47: Login Screen of the System

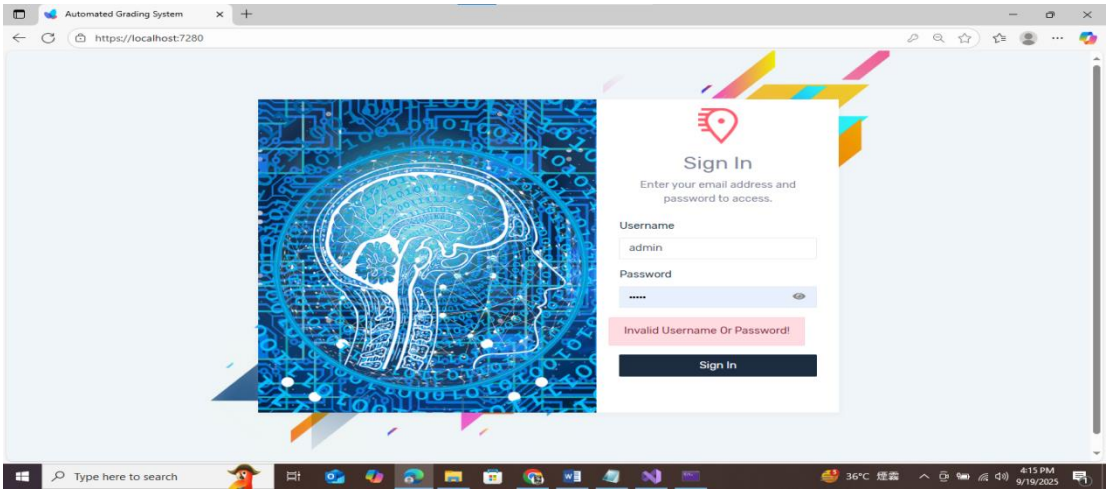


Figure 48: Example of Invalid Login Attempt

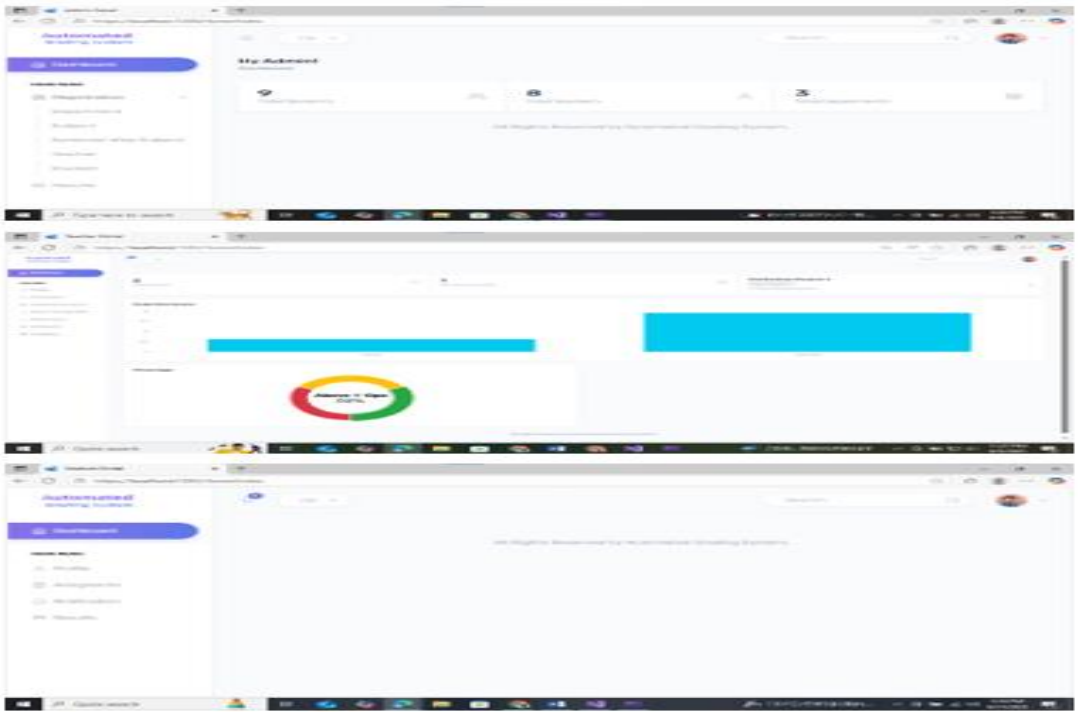


Figure 49: Successful Dashboard Access for Different User Roles (Admin, Teacher, Student)

2. Result Prediction Through NLP

The Teacher Dashboard provides multiple functionalities for monitoring and evaluating student performance.

- Graphical Results View: Teachers can visualize the predicted results of students in graphical form, making it easier to analyze overall performance trends.

- Predicted Results using NLP: The system uses Natural Language Processing (NLP) techniques to predict student results based on assignments and evaluations. This automated approach improves efficiency and ensures consistent grading.
- Feedback to Low GPA Students: Teachers can send personalized feedback messages directly to students with a GPA below 2.0, allowing them to understand their weaknesses and take corrective actions. Students can view this feedback through their own dashboards.

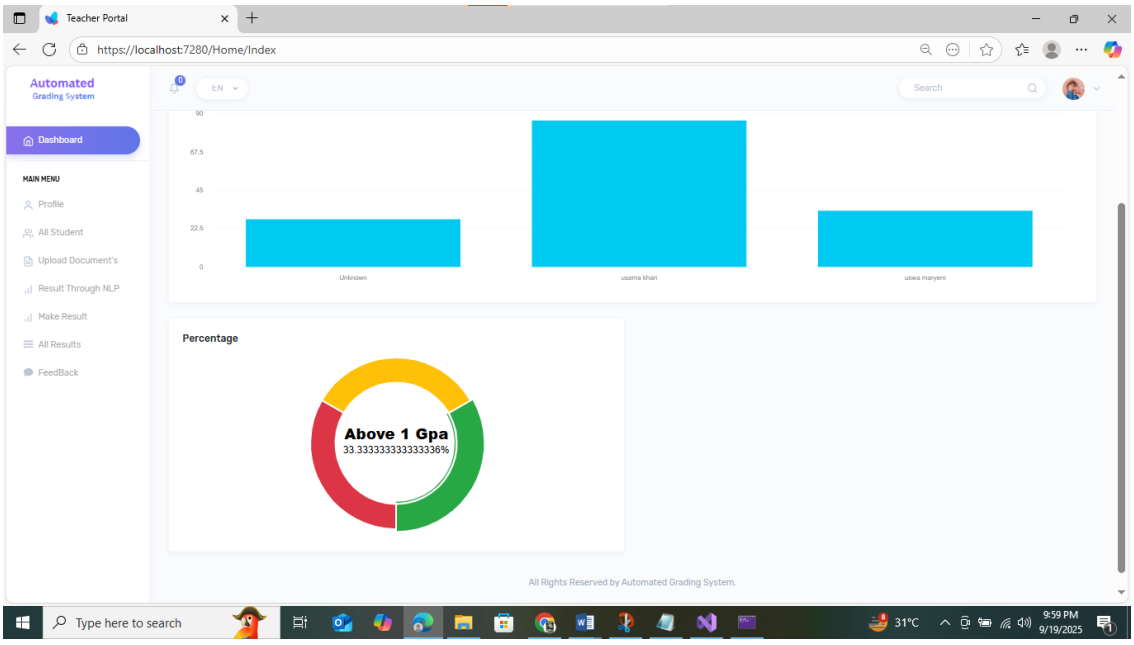


Figure 50: Graphical View and GPA Distribution Graph

Before NLP Prediction

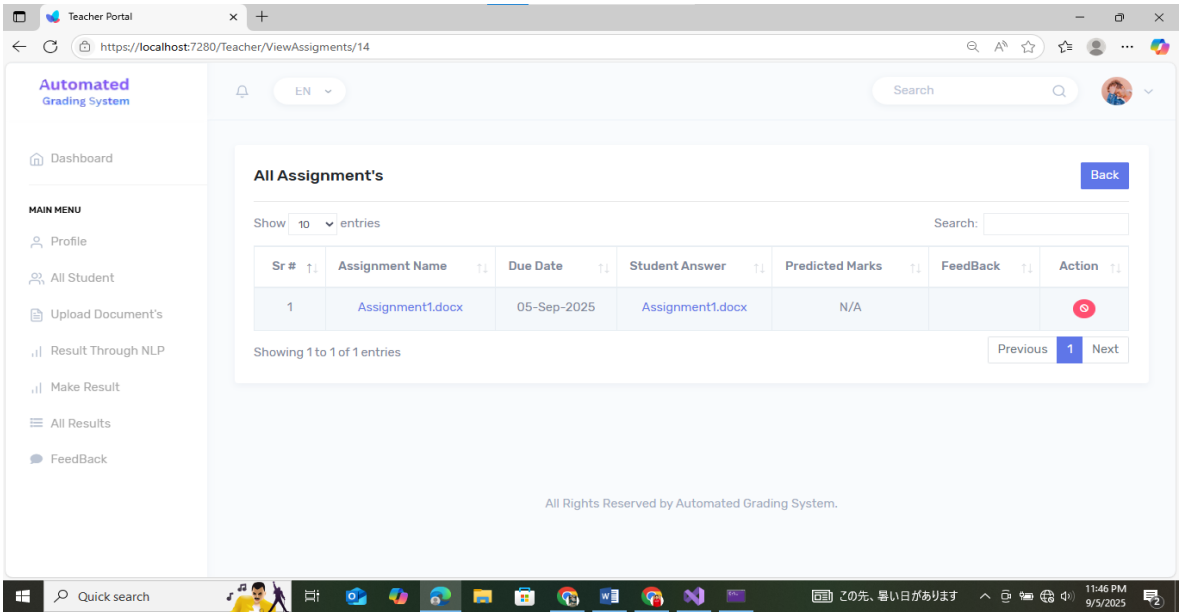


Figure 51: Student Assignment Submitted Before NLP Evaluation

After NLP Prediction

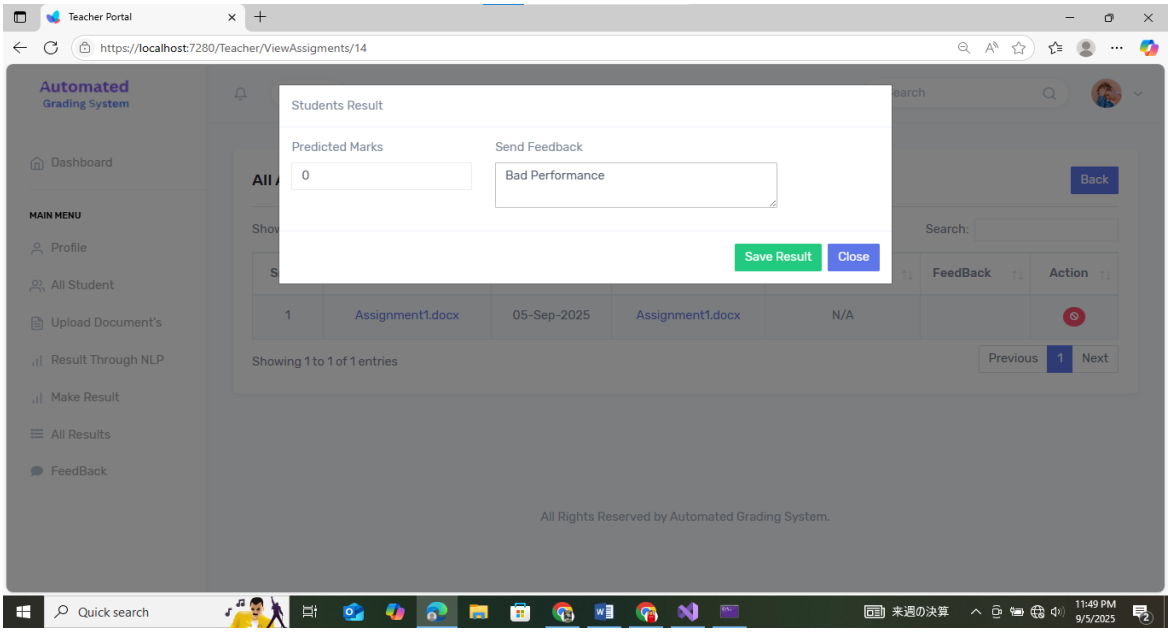


Figure 52: Assignment Result Generated Through NLP Prediction and Evaluation

Complete Result Module

After the NLP-based assignment prediction, the system integrates other evaluation components such as quiz marks, attendance marks, mid-term, and final-term marks. By combining all these factors, the system automatically generates the complete student result, including total marks, GPA, and grade.

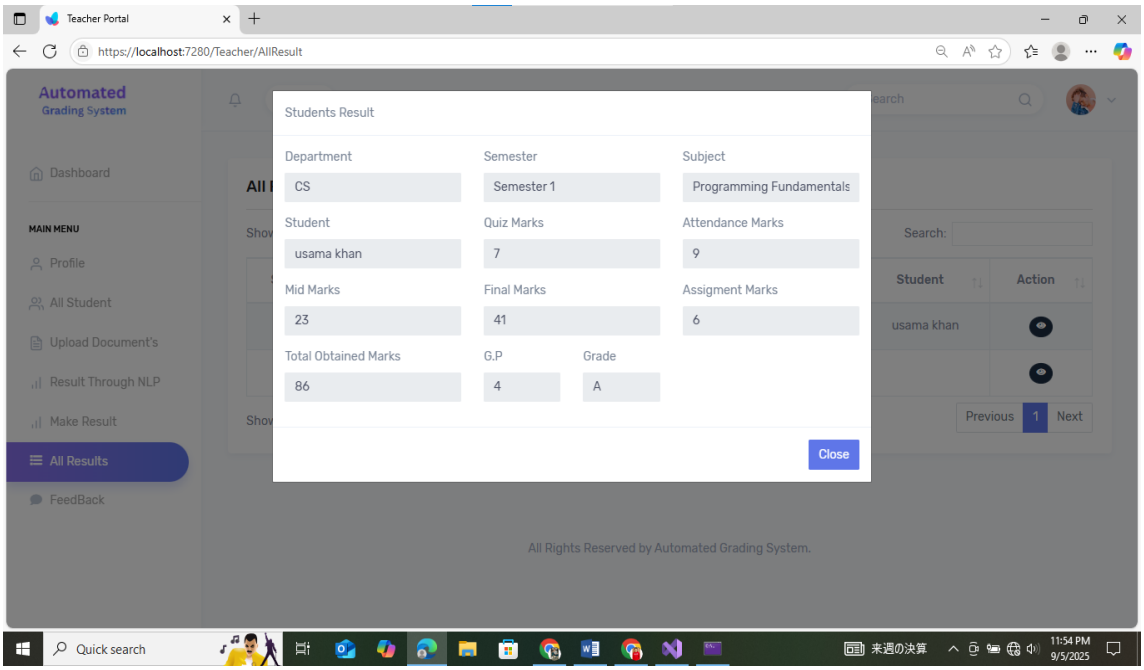


Figure 53: Complete Result View of Student

3. Notifications and Feedback

The system successfully delivers notifications and feedback messages between teachers and students.

- Teachers can send feedback to students, especially those with a GPA below 2.0.
- Students receive these notifications instantly on their dashboard and can view the details of the feedback.

The notification module ensures smooth teacher-student communication within the system.

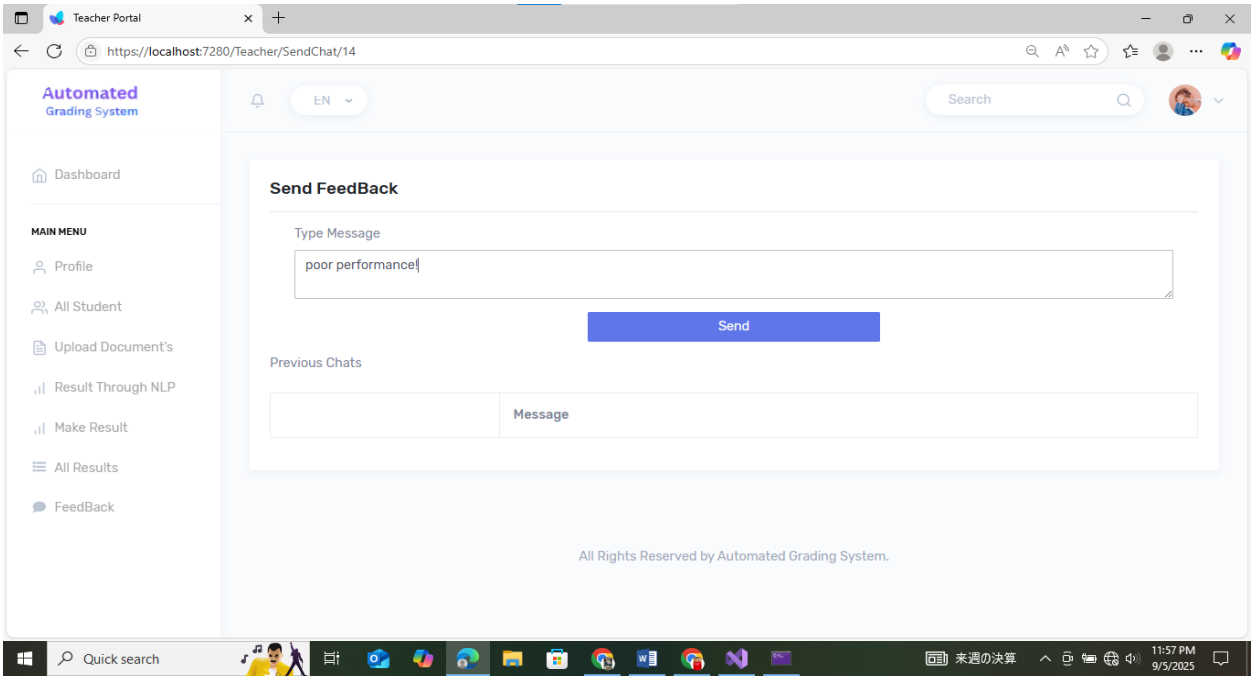


Figure 54: Teacher Feedback Sent to Student

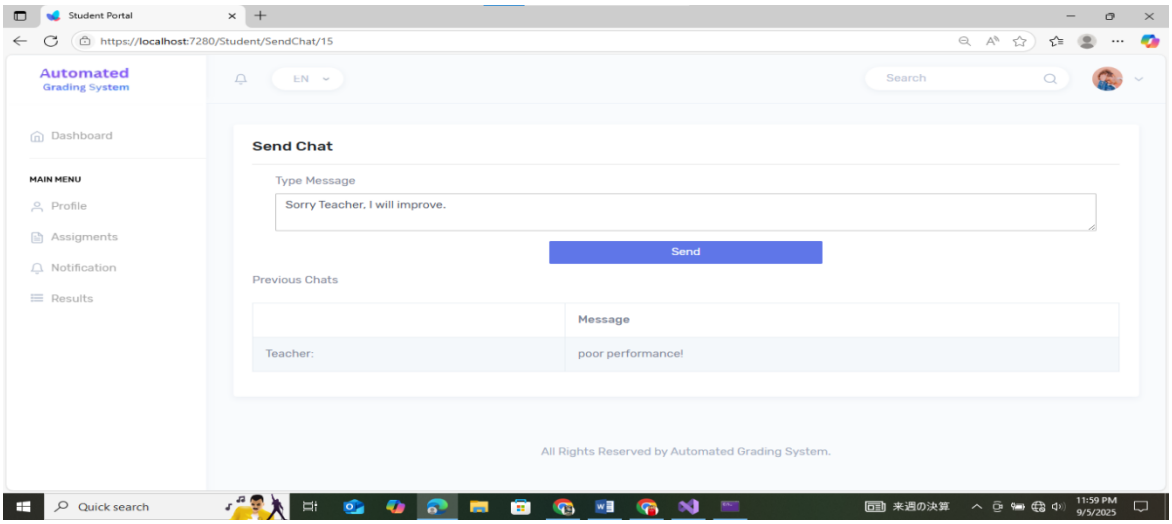


Figure 55: Result of Teacher Feedback Viewed by Student

6.2 Discussion

1. Achievement of Functional Requirements

The system successfully achieved its core functional requirements, including secure user login with role-based access (Admin, Teacher, Student), NLP-based automated assignment grading and prediction, feedback generation for low-performing students, and complete result calculation based on quizzes, assignments, attendance, midterm, and final marks. The results confirm that the

objectives of developing an intelligent, automated grading and evaluation platform have been met.

2. Strengths of the System

- Automation: Reduced manual effort in grading and result preparation.
- Accuracy: Improved result prediction accuracy using NLP and past academic data.
- Transparency: Clear feedback mechanism that helps students understand their weaknesses.
- Security: Enforced secure login with unique user accounts for multiple roles.
- User-Friendly Dashboards: Separate dashboards for teachers, students, and admin ensured clarity and accessibility.

3. Challenges and Limitations

- During implementation, some challenges were faced:
- NLP module initially struggled with ambiguous wording in assignments, requiring dataset refinement.
- Managing multiple access roles occasionally created permission conflicts, which were resolved through a centralized role management system.
- Duplicate records and invalid entries were found during student registration but were later fixed by applying input validation and database constraints.

4. Implications for Academic Institutions

The system aligns with modern educational requirements by automating OBE (Outcome-Based Education) processes and supporting quality assurance. With predictive analytics and feedback features, institutions can better monitor student performance and provide timely interventions. This directly supports compliance with accreditation standards like the Seoul Accord and enhances transparency in grading and evaluation.

5. Overall Outcome

Overall, the project achieved its defined objectives by integrating NLP-powered grading, automated result prediction, and role-based dashboards. The system demonstrated that academic evaluation processes can be digitized efficiently, improving accuracy and reducing workload. Future improvements may include mobile platform integration, advanced graphical reports, and real-time performance alerts.

7. Conclusion

This project successfully developed an Automated Student Grading System that streamlines the evaluation process through secure role-based access and intelligent automation. Built with ASP.NET Core (MVC in C#) using Visual Studio 2022, a Microsoft SQL Server 2008 backend, and a web-based frontend (HTML, CSS, JavaScript), the system achieved its objectives of reducing manual grading effort, improving fairness, and providing timely feedback to students. By integrating an NLP-based grading module powered by DeepInfra, the system enables automated analysis of assignments, prediction of results, identification of at-risk students, and generation of performance summaries. It further enhances transparency by allowing students to access feedback and performance history, while teachers and administrators can monitor progress through role-specific dashboards. The project contributes to educational technology by combining automation with secure access and predictive analytics, ensuring consistency and scalability. Although the system meets its primary goals, future improvements could include enhancing NLP accuracy with domain-specific datasets, integrating plagiarism detection, adding graphical analytics dashboards, extending access via mobile applications, and incorporating AI-driven personalized recommendations to better support students. Overall, the project demonstrates that secure web technologies combined with advanced NLP can deliver a robust, scalable, and intelligent solution for academic evaluation.

8. References

1] R. Piryani, “Project student grading system,” SlideShare. [Online]. Available: https://www.slideshare.net/rpiryani/project-student-grading-system

[2] Chandigarh University, “Project report of minor project,” Studocu. [Online]. Available: https://www.studocu.com/in/document/chandigarh-university/bachelors-of-computer-applications/project-report-of-minor-project/77813476

[3] Kashipara, “Student grading system in PHP MySQL with source code.” [Online]. Available: https://www.kashipara.com/project/php/8669/student-grading-system-in-php-mysql-with-source-code

[4] M. S. Joy and M. Luck, “The BOSS system for on-line submission and assessment,” Monitor: Journal of the CTI Center for Computing, no. 10, pp. 27–29, 1998.

[5] WebCT, WebCT software and services. [Online]. Available: http://www.webct.com/products , 2001.

[6] D. L. Wright, Grading: Measuring students’ achievement in a course. Lincoln, NE, USA: Teaching and Learning Center, University of Nebraska-Lincoln, 1987.

[7] Blackboard Inc., Blackboard products. [Online]. Available: http://www.blackboard.com, 2003.

[8] B. Yaden and P. Blaine, “How to Wiki in Moodle or Sakai: Collaboration for online language learning,” The IALLT Journal, vol. 39, no. 1, 2007.

[9] R. Laws, “Distance learning's explosion on the Internet,” Journal of Computing in Higher Education, vol. 7, no. 2, pp. 48–64, 1996.

[10] W. L. Johnson, T. Blake, and E. Shaw, “Automated management and delivery of distance courseware,” in Proc. WebNet 96, San Francisco, CA, USA, Oct. 15–19, 1996.

[11] DeepInfra, “DeepInfra API,” Deep Infra Documentation. [Online]. Available: https://deepinfra.com/docs/deep_infra_api

[12] DeepInfra, “Model Features,” Deep Infra Documentation. [Online]. Available: https://deepinfra.com/docs/model-features

[13] OpenRouter / DeepInfra Provider, “DeepInfra Provider,” OpenRouter.ai. [Online]. Available: https://openrouter.ai/provider/deepinfra

[14] I. D. Mardini, C. G. Quintero M., C. A. Viloria Núñez, W. Percybrooks, H. S. Robles N.,

and K. Villalba R., “A deep-learning-based grading system (ASAG) for reading comprehension assessment by using aphorisms as open-answer-questions,” Education and Information Technologies, vol. 29, pp. 4565–4590, 2024.
[15] DeepInfra, “Inference | AI Inference Platform | Deep Infra,” Deep Infra Documentation. [Online]. Available: https://deepinfra.com/docs/inference